



République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Tunis El Manar
École Nationale d'Ingénieurs de Tunis
**Département Technologies de
l'Information et de la Communication**

Projet de fin d'année II

**WAF intelligent de nouvelle
génération : NovaGuard AI**

Elaboré par :

Hammoudi Louay

Encadrés par :

Encadrant académique

Encadrant professionnel

Mme. Bouzouita Ines

M. Ouled Ahmed Mondher

Organisme d'accueil :



Classe : 2ème Année TEL

Année universitaire : 2025/2026

REMERCIEMENTS

On tient à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année.

On adresse nos sincères remerciements à la direction de **Sopra HR Software Tunisie** pour nous avoir accueillis au sein de son équipe ProdOps et pour avoir mis à notre disposition les ressources nécessaires au bon déroulement de ce projet.

On remercie tout particulièrement notre encadrant en entreprise, **M. OU-LED AHMED MONDHER**, pour sa disponibilité, ses précieux conseils et son suivi rigoureux tout au long de cette expérience professionnelle enrichissante.

Nos remerciements vont également à notre encadrante académique, **Mme BOUZOUITA INES**, pour ses orientations pédagogiques et ses encouragements constants.

On remercie également l'ENIT pour la qualité de sa formation, qui nous a fourni des bases solides pour conduire ce projet.

Enfin, on tient à remercier nos familles et nos amis pour leur soutien moral indéfectible tout au long de cette période de travail intense.

Résumé

Les équipes de sécurité de Sopra HR Software font face à un défi croissant : consolider et auditer les configurations de plusieurs instances ModSecurity distribuées sur l'infrastructure Apache, sans outil centralisé ni tableau de bord unifié. Ce travail manuel, chronophage et source d'erreurs, ralentit la détection d'anomalies et la réaction aux incidents.

Ce rapport présente **NovaGuard AI**, un Web Application Firewall (WAF) intelligent de nouvelle génération, développé dans le cadre d'un projet de fin d'année au sein de Sopra HR Software Tunisie. La solution repose sur une architecture trois-tiers : un *agent* autonome déployé sur les serveurs WAF (Python 3.9+), une *API centralisée* (FastAPI / OKD) chargée de la réception, validation et persistance des données, et un *frontend moderne* (Angular 17) offrant une interface d'analyse et de gouvernance. L'intelligence artificielle est intégrée via un service externe avec un fallback local par approche RAG, permettant de générer des recommandations de sécurité personnalisées.

Mots-clés : WAF, ModSecurity, FastAPI, Angular, OKD, IA, RAG, cyber-sécurité, audit de configuration.

Abstract

NovaGuard AI is a next-generation intelligent Web Application Firewall developed at Sopra HR Software Tunisia to centralize WAF governance. The solution combines an autonomous *agent* deployed on WAF servers, a centralized *API* built with FastAPI and deployed on OKD, and an Angular 17 *frontend* for monitoring and analysis. It addresses the limitations of manual ModSecurity audits by automating data collection, centralizing configuration visibility, and supporting faster incident response. An AI module, backed by an external service with a local RAG fallback, provides contextual security recommendations and helps analysts improve ModSecurity configuration

TABLE DES MATIÈRES

Remerciements	i
Liste des figures	vii
Liste des tableaux	viii
Liste des abréviations	ix
Introduction générale	1
1 Cadre du projet	3
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Présentation de Sopra Steria	3
1.1.2 Présentation de Sopra HR Software	5
1.1.3 Domaines d'activités de Sopra HR Software	5
1.1.4 Service d'accueil	5
1.2 Présentation du projet	6
1.2.1 Problématique	6
1.2.2 Étude de l'existant	6
1.2.3 Critique de l'existant	7
1.2.4 Solution proposée	7
1.3 Méthodologie de travail	8
1.3.1 Présentation des méthodologies	8
1.3.2 Comparaison entre Agile et Cascade	8
Conclusion	9

TABLE DES MATIÈRES

2	Analyse des besoins et conception	11
	Introduction	11
2.1	Expression des besoins	11
2.1.1	Périmètre et hypothèses	11
2.1.2	Parties prenantes	12
2.2	Besoins fonctionnels	12
2.2.1	Cas d’usage principaux	12
2.2.2	Diagramme de cas d’utilisation	13
2.3	Besoins non fonctionnels	13
2.4	Architecture globale	14
2.5	Architecture détaillée	15
2.6	Modèles de données	16
2.7	Planification du projet	16
2.7.1	Phases du projet	16
2.7.2	Diagramme de Gantt	17
2.7.3	Calendrier prévisionnel	18
	Conclusion	19
3	Étude théorique	21
	Introduction	21
3.1	Web Application Firewall (WAF)	21
3.1.1	Modes de déploiement	22
3.1.2	ModSecurity	22
3.1.3	Principales attaques couvertes	23
3.1.4	Limites des WAF traditionnels	24
3.1.5	Comparaison des solutions WAF	24
3.2	Technologies backend	26
3.2.1	FastAPI	26
3.2.2	PostgreSQL	28
3.2.3	OKD – Origin Kubernetes Distribution	28
3.3	Technologies frontend	28
3.3.1	Angular 17	29
3.4	Intelligence Artificielle appliquée à la sécurité	29
3.4.1	Retrieval-Augmented Generation (RAG)	29
3.4.2	Exécution locale avec Ollama (Mistral)	30
3.4.3	Détection d’anomalies	30
3.5	Sécurité des communications	30
3.5.1	Keycloak	30
3.5.2	TLS et vérification SSL	31
	Conclusion	31

TABLE DES MATIÈRES

4	Réalisation	32
	Introduction	32
4.1	Réalisation par phase	32
4.1.1	Phase 1 : Analyse des besoins	32
4.1.2	Phase 2 : Conception	33
4.1.3	Phase 3 : Développement	33
4.1.4	Phase 4 : Tests et validation	34
4.1.5	Phase 5 : Déploiement et documentation	34
4.2	Agent NovaGuard sur RHEL	35
4.2.1	Rôle de l'agent	35
4.2.2	Architecture interne de l'agent	35
4.2.3	Structure fonctionnelle implémentée	36
4.2.4	Supervision des logs ModSecurity	37
4.2.5	Normalisation des données	38
4.2.6	Configuration réelle utilisée	39
4.2.7	Robustesse et gestion d'erreurs	39
4.2.8	Difficultés rencontrées côté agent	39
4.2.9	Déploiement et planification sur RHEL	40
4.3	API WAF Backend	41
4.3.1	Choix techniques	41
4.3.2	Flux interne de traitement d'un payload	41
4.3.3	Modèles de données SQLAlchemy	42
4.3.4	Validation du payload avec Pydantic	42
4.3.5	Étapes de développement de l'API	42
4.3.6	Endpoints REST implémentés	43
4.3.7	Endpoint de consultation des logs	43
4.3.8	Difficultés rencontrées côté API	44
4.4	Frontend Angular	44
4.4.1	Architecture des composants	44
4.4.2	Pages et modules réalisés	45
4.4.3	Page Logs : affichage des événements ModSecurity	47
4.4.4	Difficultés rencontrées côté frontend	48
4.4.5	Étapes de développement	48
4.4.6	UX et éléments visuels	49
4.5	Module d'Intelligence Artificielle	49
4.5.1	Approche hybride	49
4.5.2	Cas d'usage implémentés	49
4.5.3	Exemple de réponse IA intégrée	50
4.6	Sécurité de la solution	50
	Conclusion	51

TABLE DES MATIÈRES

5 Tests et analyse des résultats	53
Introduction	53
5.1 Stratégie de tests	53
5.1.1 Tests de l'agent	53
5.1.2 Tests de l'API	54
5.1.3 Tests de l'interface utilisateur	55
5.1.4 Environnement de test	56
5.1.5 Critères de validation retenus	57
5.1.6 Analyse qualitative des campagnes de test	57
5.2 Résultats obtenus	58
5.2.1 Test de bout en bout	58
5.2.2 Interprétation des résultats par composant	58
5.2.3 Synthèse des bénéfices observés	59
5.2.4 Limites de l'évaluation	60
Conclusion	60
Conclusion générale	61
Références bibliographiques	62
Annexes	64
A Arborescence du projet	65
B Configuration complète de l'agent	66
C Commandes d'exécution de l'agent	67
D Configuration recommandée en production	68
E Guide de dépannage de l'agent	69

TABLE DES FIGURES

1.1	Répartition du chiffre d'affaires de Sopra Steria au 31 décembre 2022	4
1.2	Modèle en cascade adopté pour le projet NovaGuard	9
2.1	Diagramme de cas d'utilisation de NovaGuard	13
2.2	Architecture globale de NovaGuard	15
2.3	Diagramme de Gantt du projet NovaGuard	18
3.1	Architecture en couches de FastAPI dans NovaGuard	27
4.1	Journalisation de l'agent NovaGuard sur RHEL	33
4.2	Dashboard de gouvernance WAF – vue générale	34
4.3	Événement normalisé extrait de modsec_audit.log	38
4.4	Diagramme de séquence – traitement d'un payload par l'API .	41
4.5	Architecture des composants Angular du frontend NovaGuard	45
4.6	Page Dashboard du frontend NovaGuard	45
4.7	Page Logs – consultation des événements d'audit ModSecurity	46
4.8	Vue frontend de gestion des règles WAF	46
4.9	Création d'une règle personnalisée depuis le frontend	47
4.10	Affichage de l'analyse IA dans le frontend NovaGuard	50

LISTE DES TABLEAUX

1.1	Domaines d'activités de Sopra HR Software	5
1.2	Comparaison entre l'approche en Cascade et l'approche Agile	8
2.1	Description des composants de NovaGuard	16
2.2	Phases du projet NovaGuard	17
2.3	Calendrier prévisionnel et livrables	18
3.1	Modes de déploiement d'un WAF	22
3.2	Sections du journal d'audit ModSecurity	23
3.3	Comparaison des principales solutions WAF	25
4.1	Composants internes de l'agent NovaGuard	36
4.2	Paramètres de configuration liés à la supervision des logs	37
4.3	Modèles de données SQLAlchemy de NovaGuard	42
4.4	Endpoints REST de l'API WAF NovaGuard	43
4.5	Colonnes affichées dans la page Logs du frontend	47
4.6	Mécanismes de sécurité de la solution NovaGuard	51
5.1	Tests de l'agent NovaGuard	54
5.2	Tests de l'API WAF Backend	55
5.3	Tests du frontend Angular	56
5.4	Synthèse des bénéfices observés lors des tests	59
E.1	Tableau de dépannage de l'agent NovaGuard	69

LISTE DES ABRÉVIATIONS

Sigle	Signification
WAF	Web Application Firewall
API	Application Programming Interface
IA	Intelligence Artificielle
RAG	Retrieval-Augmented Generation
OKD	Origin Kubernetes Distribution
TLS	Transport Layer Security
REST	Representational State Transfer
JSON	JavaScript Object Notation
SOC	Security Operations Center
OWASP	Open Web Application Security Project
CRS	Core Rule Set (ensemble de règles OWASP)
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
JWT	JSON Web Token
SSO	Single Sign-On
ENIT	École Nationale d'Ingénieurs de Tunis
PFA	Projet de Fin d'Année
SSII	Société de Services en Ingénierie Informatique
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
RHEL	Red Hat Enterprise Linux

INTRODUCTION GÉNÉRALE

Face à l'évolution constante du paysage des cybermenaces, la sécurité des applications web est devenue un enjeu stratégique majeur pour toute entreprise numérique. Les Web Application Firewalls (WAF) constituent la première ligne de défense contre les attaques applicatives telles que les injections SQL, les scripts intersites (XSS) ou les traversées de répertoire.

Au sein de Sopra HR Software, l'infrastructure de production s'appuie sur ModSecurity, un WAF open-source robuste mais dont la gestion distribuée génère une charge opérationnelle importante. En l'absence d'un outil centralisé, les équipes de sécurité doivent auditer manuellement chaque serveur, ce qui engendre des délais de réaction inacceptables lors d'incidents.

C'est dans ce contexte que s'inscrit le projet **NovaGuard AI**, dont l'objectif est de concevoir et développer une plateforme unifiée de gouvernance WAF, pilotée par l'intelligence artificielle, capable d'automatiser la collecte de données, de détecter les écarts de configuration et de générer des recommandations actionnables en temps réel.

Ce rapport est structuré en cinq chapitres :

- **Chapitre 1 : Cadre du projet**
- **Chapitre 2 : Analyse des besoins et conception**
- **Chapitre 3 : Étude théorique**
- **Chapitre 4 : Réalisation**
- **Chapitre 5 : Tests et analyse des résultats**

Une conclusion générale clôt ce rapport en synthétisant les apports du projet et en proposant des perspectives d'évolution.

Chapitre 1 : Cadre du projet

Sommaire du chapitre

1.1 Présentation de l'organisme d'accueil	3
1.2 Présentation du projet.....	6
1.3 Méthodologie de travail.....	8

CHAPITRE 1

CADRE DU PROJET

Introduction

Dans ce premier chapitre, on introduit l'organisme d'accueil au sein duquel on a effectué ce stage. Ensuite, on met en évidence la problématique tout en critiquant constructivement l'existant. Enfin, on présente la solution proposée ainsi que la méthodologie de travail retenue.

Avant d'aborder la problématique technique du projet, il est nécessaire de présenter le contexte organisationnel dans lequel il a été réalisé. Cette présentation permet de mieux comprendre l'environnement professionnel, les activités de l'entreprise d'accueil ainsi que le rôle du service concerné.

1.1 Présentation de l'organisme d'accueil

Dans cette section, on présente l'organisme d'accueil, son positionnement dans son secteur d'activité ainsi que l'environnement professionnel dans lequel ce projet a été réalisé.

1.1.1 Présentation de Sopra Steria

Sopra Steria est un leader européen de la transformation digitale qui propose l'un des portefeuilles d'offres les plus complets du marché : conseil, intégration de systèmes, édition de solutions métier, infrastructure management et business process services. Il apporte ainsi une réponse globale aux enjeux de

développement et de compétitivité des grandes entreprises et organisations. Combinants valeur ajoutée, innovation et performance des services délivrés, Sopra Steria accompagne ses clients dans leur transformation et les aide à faire le meilleur usage du numérique [13].

Répartition du chiffre d'affaires au 31 décembre 2022

La répartition du chiffre d'affaires de Sopra Steria au 31 décembre 2022 peut être présentée selon trois axes : géographique, par activité et par secteur.

Répartition par activité :

- Conseil et intégration des systèmes : 62%
- Hybrid Cloud et Technology Services : 10%
- Édition de solutions métiers : 15%
- Business Process Services : 13%

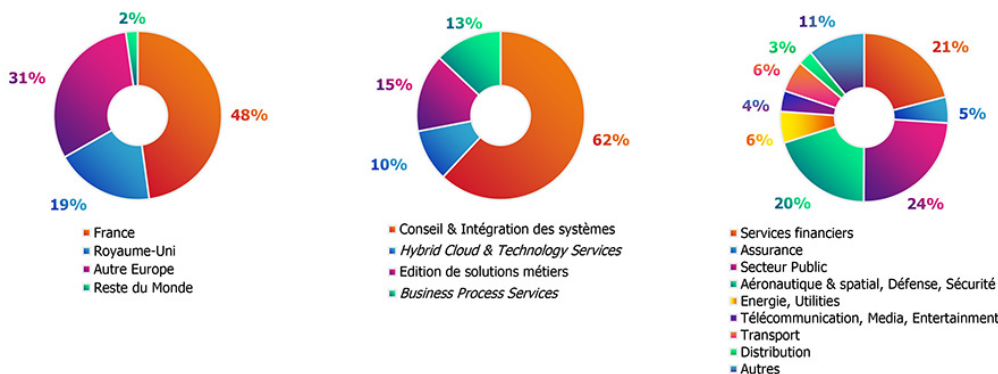


FIGURE 1.1 – Répartition du chiffre d'affaires de Sopra Steria au 31 décembre 2022

Sopra Steria : un leader européen de la transformation numérique

Sopra Steria se positionne parmi les principaux acteurs du marché numérique en Europe, avec une présence forte sur les services numériques et la transformation des organisations.

- Top 5 des acteurs européens du secteur des ESN
- Top 12 des acteurs œuvrant en Europe
- 5,6 Mrd € de chiffre d'affaires en 2025
- 51 000 collaborateurs dans le monde

1.1.2 Présentation de Sopra HR Software

Sopra HR Software est une filiale du groupe français Sopra Steria. Elle édite des solutions RH parfaitement adaptées aux besoins des Directions des Ressources Humaines et aux organisations de moyennes et grandes tailles, des secteurs public et privé. Sopra HR Software accompagne plus de 850 clients dans plus de 54 pays, en mode *on-premise* ou services d'outsourcing. Elle est présente dans 20 pays [13].

1.1.3 Domaines d'activités de Sopra HR Software

Sopra HR Software intervient dans plusieurs domaines clés de la gestion des ressources humaines, tels que représentés dans le tableau 1.1 ci-dessous.

Domaine	Description
Gestion des temps et activités	Planification, suivi des présences, gestion des absences
Gestion des talents	Recrutement, formation, évaluation des compétences
Espace collaboratif	Portail self-service et outils de communication interne
Pilotage et performance	Tableaux de bord RH et aide à la décision
Gestion administrative et paie	Traitement de la paie, déclarations sociales, conformité

TABLE 1.1 – Domaines d'activités de Sopra HR Software

1.1.4 Service d'accueil

Mon stage de fin d'année a été réalisé au sein de l'entité **Outsourcing Services** dont les activités représentent la tendance sur le marché en termes de sous-traitance de l'activité RH. On a intégré l'équipe **ProdOps** (Production Operations), dont les principales fonctions sont :

- La réception des demandes des clients et leur communication aux différentes équipes.
- L'administration des environnements, le transfert des données et la migration technique.
- L'installation des applications.
- La gestion des projets *Outsourcing*.
- La gestion des plateformes.

Après avoir présenté l'organisme d'accueil et le cadre professionnel du stage, il convient maintenant de situer le projet réalisé dans ce contexte. Cette section présente donc la problématique rencontrée, l'existant étudié ainsi que la solution proposée pour y répondre.

1.2 Présentation du projet

Le présent projet, intitulé *NovaGuard AI – WAF Intelligent de Nouvelle Génération*, a été effectué dans le cadre de ce projet de fin d'année au sein de Sopra HR Software Tunisie, en vue de l'obtention du diplôme national d'ingénieur en Télécommunications à l'ENIT. L'objectif principal a été de concevoir et développer une plateforme centralisée de gouvernance WAF, pilotée par l'intelligence artificielle, capable de remplacer la gestion manuelle et fragmentée de ModSecurity actuellement en place.

1.2.1 Problématique

Face à l'évolution constante des menaces en ligne, on s'est confronté à une problématique concrète : les équipes sécurité de Sopra HR doivent consolider l'état des instances ModSecurity actives, les règles chargées et les configurations sur chaque serveur Apache. Les audits sont souvent manuels et épars, ce qui ralentit considérablement la détection d'anomalies et la réaction aux incidents.

On a identifié plusieurs failles structurelles dans le système actuel :

- **Absence de centralisation** : chaque serveur est audité indépendamment, sans vision globale de la posture de sécurité.
- **Gestion manuelle des logs** : l'exploitation des logs ModSecurity est fastidieuse et peu reproductible.
- **Taux élevé de faux positifs** : les règles statiques génèrent de nombreuses alertes non pertinentes, surchargeant les équipes.
- **Absence d'automatisation** : aucune réponse automatique aux menaces détectées n'est mise en place.

1.2.2 Étude de l'existant

La solution en place pour assurer la sécurité des applications web chez Sopra HR repose exclusivement sur **ModSecurity** (v2/v3) configuré en mode embarqué sur les serveurs Apache. Bien que ModSecurity soit un WAF reconnu et mature [2], la solution actuelle présente des lacunes significatives dans le contexte d'une infrastructure distribuée :

- Absence d’outil de collecte centralisée des configurations et règles actives.
- Méthodes d’audit non standardisées, potentiellement incohérentes d’un serveur à l’autre.
- Aucune intégration avec un tableau de bord de visualisation ou d’analyse.
- Pas d’interface permettant de définir ou personnaliser des règles de sécurité à distance.

1.2.3 Critique de l’existant

L’analyse de la solution existante a permis de mettre en évidence plusieurs points critiques. D’une part, l’absence d’un processus d’audit structuré expose potentiellement l’infrastructure à des vulnérabilités non détectées. D’autre part, la surcharge des équipes sécurité due à la gestion manuelle des logs et des configurations compromet la réactivité en cas d’incident.

La rigidité du système de règles statiques de ModSecurity, sans mécanisme d’adaptation intelligente, génère par ailleurs un taux élevé de faux positifs, nuisant à la productivité des équipes et à la disponibilité des applications.

1.2.4 Solution proposée

Pour répondre à ces problèmes, on a proposé et développé **NovaGuard AI**, une plateforme de gouvernance WAF de nouvelle génération reposant sur trois composants principaux :

1. **Agent NovaGuard** : un agent autonome en Python 3.9+, déployé sur chaque serveur WAF, qui collecte automatiquement les configurations ModSecurity, les règles actives et les logs d’audit, puis envoie le tout à l’API centrale via un payload JSON normalisé.
2. **API WAF Backend** : une API REST développée avec FastAPI [3], déployée sur OKD, qui reçoit, valide et persiste les données dans une base PostgreSQL, et expose des endpoints pour le frontend.
3. **Frontend NovaGuard** : une interface Angular 17 [4] permettant de visualiser par serveur les règles et configurations actives, de consulter les analyses IA et de générer des règles personnalisées.

L’intelligence artificielle, intégrée via un service externe avec fallback local par approche RAG [10], permet de produire des recommandations de sécurité actionnables et de proposer des règles custom adaptées au contexte de chaque serveur.

Une fois la problématique identifiée et la solution proposée définie, il est important de préciser la démarche suivie pour organiser la réalisation du projet. La section suivante présente ainsi la méthodologie de travail retenue et justifie son adéquation avec les objectifs fixés.

1.3 Méthodologie de travail

Dans cette section, on présente la démarche méthodologique suivie pour structurer le projet et justifier le choix de l'approche la plus adaptée au contexte de réalisation.

1.3.1 Présentation des méthodologies

Dans le domaine de la gestion de projets informatiques, deux méthodologies principales sont fréquemment utilisées : la méthodologie **Agile** (SCRUM) et la méthodologie **en Cascade** (*Waterfall*). SCRUM divise le projet en *sprints* itératifs, favorisant l'adaptabilité. Le modèle en cascade adopte quant à lui une approche linéaire et séquentielle, exigeant que chaque phase soit complétée avant que la suivante ne démarre.

1.3.2 Comparaison entre Agile et Cascade

Caractéristique	Approche en Cascade	Approche Agile
Approche	Linéaire et séquentielle	Itérative et incrémentale
Phases	Distinctes et séquentielles	Itérations (sprints)
Documentation	Fortement documentée	Documentation adaptative
Flexibilité	Peu de flexibilité aux changements	Adaptabilité aux changements
Collaboration	Moins de collaboration	Collaboration intensive
Livraison	En fin de projet	Incrémentielle
Convient pour	Projets à exigences stables	Projets évolutifs et agiles

TABLE 1.2 – Comparaison entre l'approche en Cascade et l'approche Agile

Suite à cette comparaison, on a retenu la **méthodologie en Cascade** pour ce projet. En effet, les exigences fonctionnelles et non fonctionnelles étaient clairement définies dès le début, ce qui favorise une approche linéaire offrant

une meilleure prévisibilité des délais et une documentation détaillée à chaque étape. La figure 1.2 illustre ce modèle.

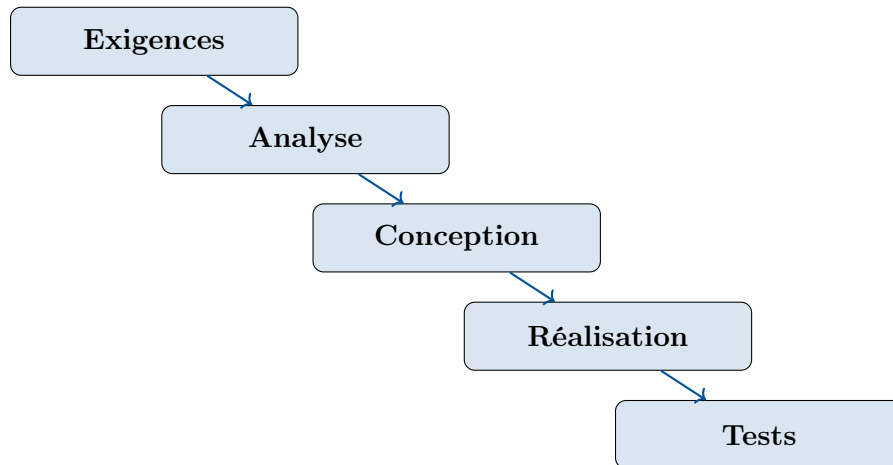


FIGURE 1.2 – Modèle en cascade adopté pour le projet NovaGuard

Conclusion

Dans ce chapitre, on a présenté l'organisme d'accueil, exposé la problématique identifiée, critiqué l'existant et proposé la solution NovaGuard AI. On a également justifié le choix de la méthodologie en cascade. Le chapitre suivant se consacre à l'analyse des besoins et à la conception de la solution.

Chapitre 2 : Analyse des besoins et conception

Sommaire du chapitre

2.1 Expression des besoins	11
2.2 Besoins fonctionnels	12
2.3 Besoins non fonctionnels	13
2.4 Architecture globale	14
2.5 Architecture détaillée	15
2.6 Modèles de données	16
2.7 Planification du projet	16

CHAPITRE 2

ANALYSE DES BESOINS ET CONCEPTION

Introduction

Ce chapitre présente l'analyse des besoins fonctionnels et non fonctionnels qu'on a identifiés pour la solution NovaGuard, ainsi que la conception architecturale qui en découle.

Après avoir défini le cadre général du projet, il est nécessaire d'identifier précisément les attentes auxquelles la solution doit répondre. L'expression des besoins constitue ainsi la première étape de l'analyse fonctionnelle et permet de délimiter le périmètre du système à concevoir.

2.1 Expression des besoins

Dans cette section, on précise les attentes du projet NovaGuard en définissant le périmètre, les contraintes et les parties prenantes. Cette expression des besoins sert de base à la spécification fonctionnelle et aux choix de conception présentés dans la suite du chapitre.

2.1.1 Périmètre et hypothèses

On a délimité le périmètre du projet aux serveurs applicatifs équipés de ModSecurity (v2/v3) au sein de l'infrastructure Sopra HR. La collecte porte

sur les fichiers de configuration et l’audit des règles actives. La collecte des événements (logs) est optionnelle selon les permissions et la volumétrie du serveur cible. L’agent suppose un accès en lecture aux fichiers ModSecurity et aux logs d’audit.

2.1.2 Parties prenantes

On a identifié les parties prenantes suivantes :

- **Équipe sécurité** : audit de configuration et recommandations.
- **Équipe exploitation** : déploiement de l’agent et supervision.
- **Équipe développement** : consultation des règles et alignement WAF.
- **Management** : suivi d’avancement et gouvernance.

Le périmètre du projet et les parties prenantes ayant été identifiés, il convient désormais de traduire ces attentes en fonctionnalités concrètes. La section suivante présente donc les besoins fonctionnels que la plateforme NovaGuard doit satisfaire.

2.2 Besoins fonctionnels

À l’issue de l’analyse menée, on a identifié les besoins fonctionnels suivants :

1. **Collecte automatisée** : l’agent collecte les fichiers de configuration ModSecurity, les règles actives et les métadonnées système.
2. **Audit des règles** : comptage et liste des règles actives par serveur.
3. **Enregistrement des sources WAF** : hostname, IP, version, dernière collecte.
4. **Visualisation** : affichage des règles et configurations par serveur via une interface web moderne.
5. **Analyse IA** : génération de recommandations de sécurité et de règles personnalisées.
6. **Authentification** : accès sécurisé via Keycloak (SSO).

2.2.1 Cas d’usage principaux

On a identifié les cas d’usage principaux suivants :

- Audit rapide d’un serveur : liste des règles et configurations actives.
- Vérification de `SecRuleEngine` et des directives critiques.
- Création d’une règle custom à partir d’une alerte détectée.
- Comparaison entre serveurs et détection d’écarts de configuration.

2.2.2 Diagramme de cas d'utilisation

Afin de représenter de manière claire les interactions entre les acteurs et le système, on a réalisé le diagramme de cas d'utilisation présenté dans la Figure 2.1. Ce diagramme met en évidence les fonctionnalités principales de la plateforme NovaGuard et les acteurs qui y ont accès.

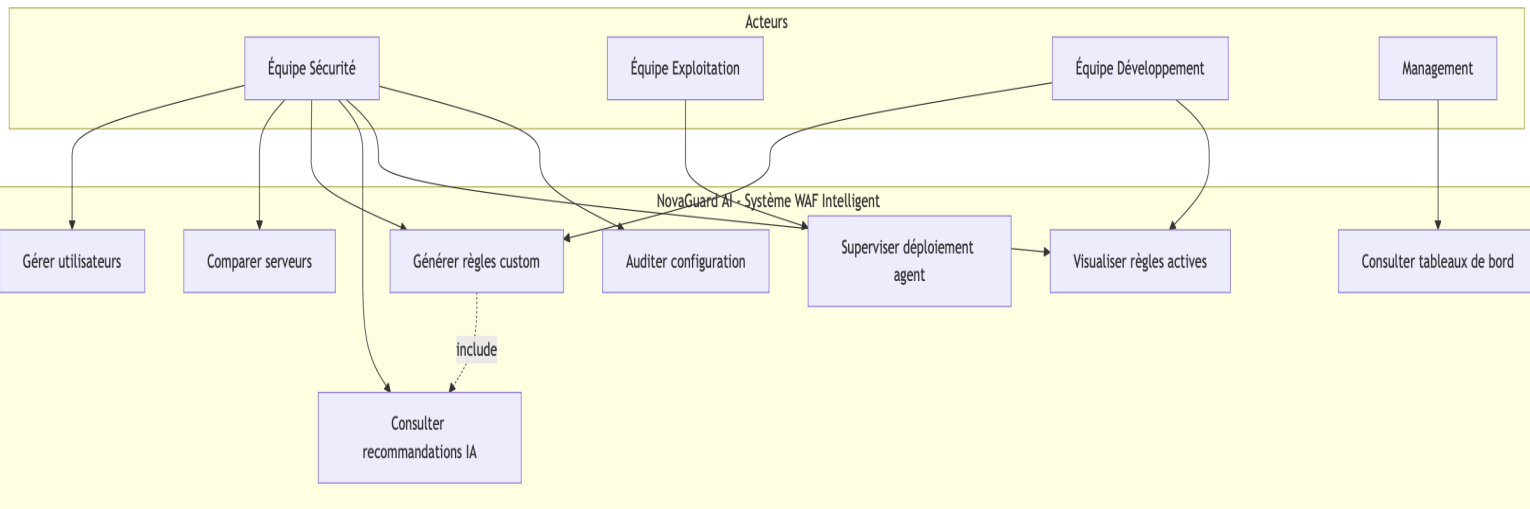


FIGURE 2.1 – Diagramme de cas d'utilisation de NovaGuard

Les besoins fonctionnels décrivent ce que le système doit réaliser du point de vue métier. Toutefois, la qualité d'une solution dépend également de contraintes transversales telles que la sécurité, la disponibilité et la performance, qui sont abordées dans la section suivante.

2.3 Besoins non fonctionnels

En complément des besoins fonctionnels, on a défini les contraintes non fonctionnelles suivantes :

- **Disponibilité** : tolérance aux pannes agent et haute disponibilité de l'API.
- **Scalabilité** : support de plusieurs agents en parallèle.
- **Sécurité** : trafic chiffré TLS pour toutes les communications.

- **Performance** : temps de réponse inférieur à 2 secondes pour la consultation.
- **Observabilité** : logs de collecte et traces d’erreurs côté agent et API.

Après avoir défini les exigences fonctionnelles et non fonctionnelles, il devient possible de proposer une organisation technique adaptée. La section suivante présente donc l’architecture globale retenue pour structurer les différents composants de NovaGuard.

2.4 Architecture globale

L’architecture de NovaGuard repose sur quatre couches complémentaires : l’agent NovaGuard (Python) collecte les données et les envoie à l’API Backend (FastAPI) qui les stocke dans PostgreSQL. Le Frontend (Angular) consomme ces données pour l’affichage. Le Service IA analyse les configurations et génère des recommandations de sécurité. Cette architecture est représentée dans la figure 2.2.

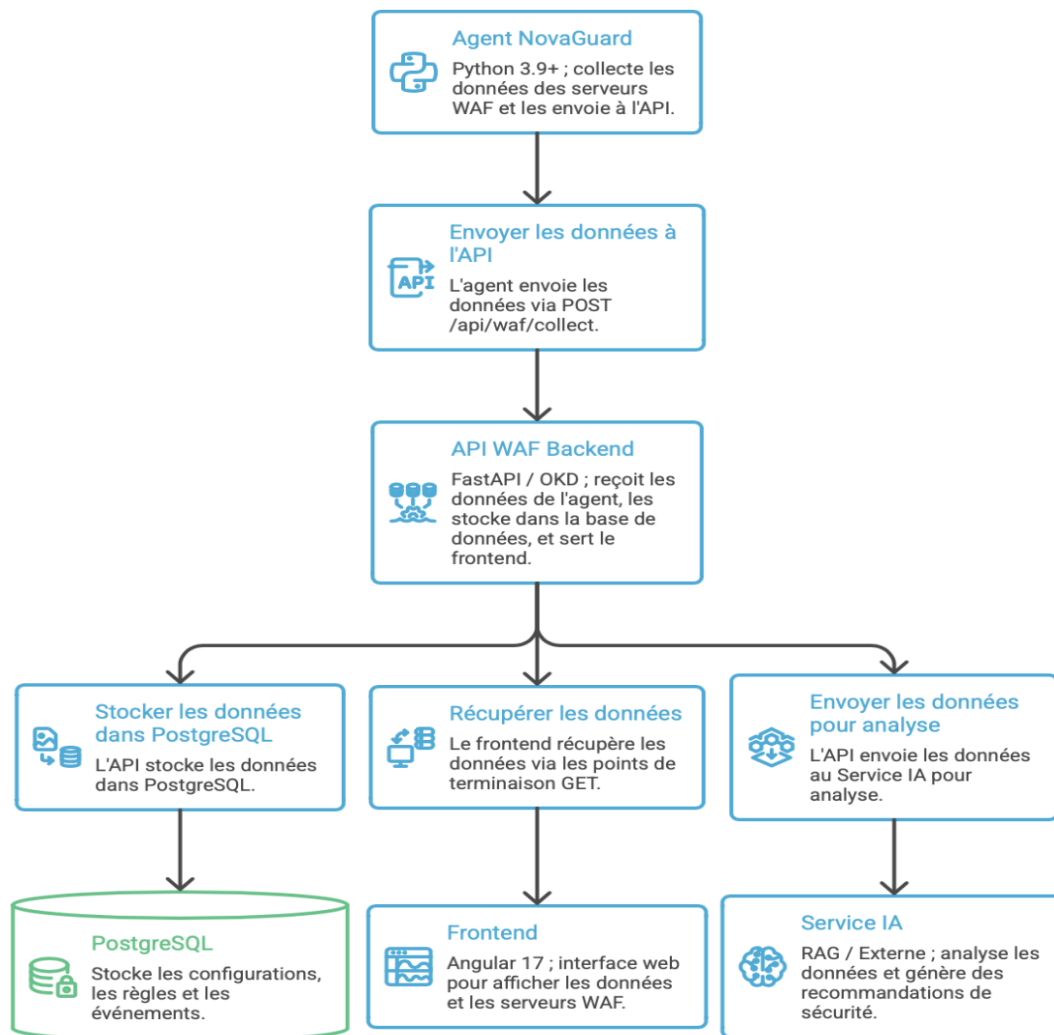


FIGURE 2.2 – Architecture globale de NovaGuard

L'architecture globale permet d'avoir une vision d'ensemble des principaux blocs de la solution. Afin de mieux comprendre le rôle de chaque composant, il est nécessaire d'en détailler le fonctionnement et les responsabilités respectives.

2.5 Architecture détaillée

Le tableau 2.1 décrit le rôle de chaque composant de la solution.

Composant	Rôle
Agent	Collecte configs, règles, logs et métadonnées système
API WAF	Validation du payload, persistance, endpoints REST
Base de données	Stockage des sources, règles actives, configs, events
Frontend	Interface WAF par serveur, modal configs, analyse IA
Auth (Keycloak)	Gestion de l'accès utilisateur (SSO / JWT)

TABLE 2.1 – Description des composants de NovaGuard

Les composants de l'architecture échangent et manipulent plusieurs types d'informations liés aux serveurs WAF, aux configurations et aux événements de sécurité. Il est donc essentiel de définir les modèles de données qui assurent la cohérence et la persistance de ces informations.

2.6 Modèles de données

On a défini les principaux modèles de données persistées suivants :

- **Source WAF** : `hostname`, `ip`, `version`, `last_seen`.
- **Rules audit** : liste des règles actives et compteur par serveur.
- **Config files** : chemins, tailles, dates de modification, contenu.
- **Events** : transactions, sévérité, action effectuée, IP client.
- **Custom rules** : titre, texte de la règle, mode, rapport IA associé.

Une fois les besoins, l'architecture et les modèles de données définis, la réalisation du projet doit être organisée dans le temps. La section suivante présente la planification retenue ainsi que les principales phases de développement.

2.7 Planification du projet

Cette section présente l'organisation temporelle du projet NovaGuard, en détaillant les phases de réalisation, leur enchaînement et la répartition des tâches. Elle permet de justifier la démarche suivie pour respecter les objectifs et les délais fixés.

2.7.1 Phases du projet

Conformément à la méthodologie en cascade adoptée, on a structuré le projet en cinq phases principales, détaillées dans le tableau 2.2.

Phase	Description
Phase 1 : Analyse des besoins	Recueil et formalisation des besoins fonctionnels et non fonctionnels, étude de l'existant, rédaction du cahier des charges
Phase 2 : Conception	Conception architecturale, modélisation UML, définition des modèles de données, choix technologiques
Phase 3 : Développement	Développement de l'agent Python, de l'API FastAPI, du frontend Angular et intégration de l'IA
Phase 4 : Tests	Tests unitaires, d'intégration, de bout en bout, validation des performances et de la sécurité
Phase 5 : Déploiement et documentation	Déploiement sur OKD, rédaction de la documentation technique et utilisateur, livraison finale

TABLE 2.2 – Phases du projet NovaGuard

2.7.2 Diagramme de Gantt

Pour visualiser la répartition des tâches dans le temps et les dépendances entre elles, on a réalisé un diagramme de Gantt présenté dans la figure 2.3. Ce diagramme illustre le déroulement du projet sur une durée totale de 9 semaines.

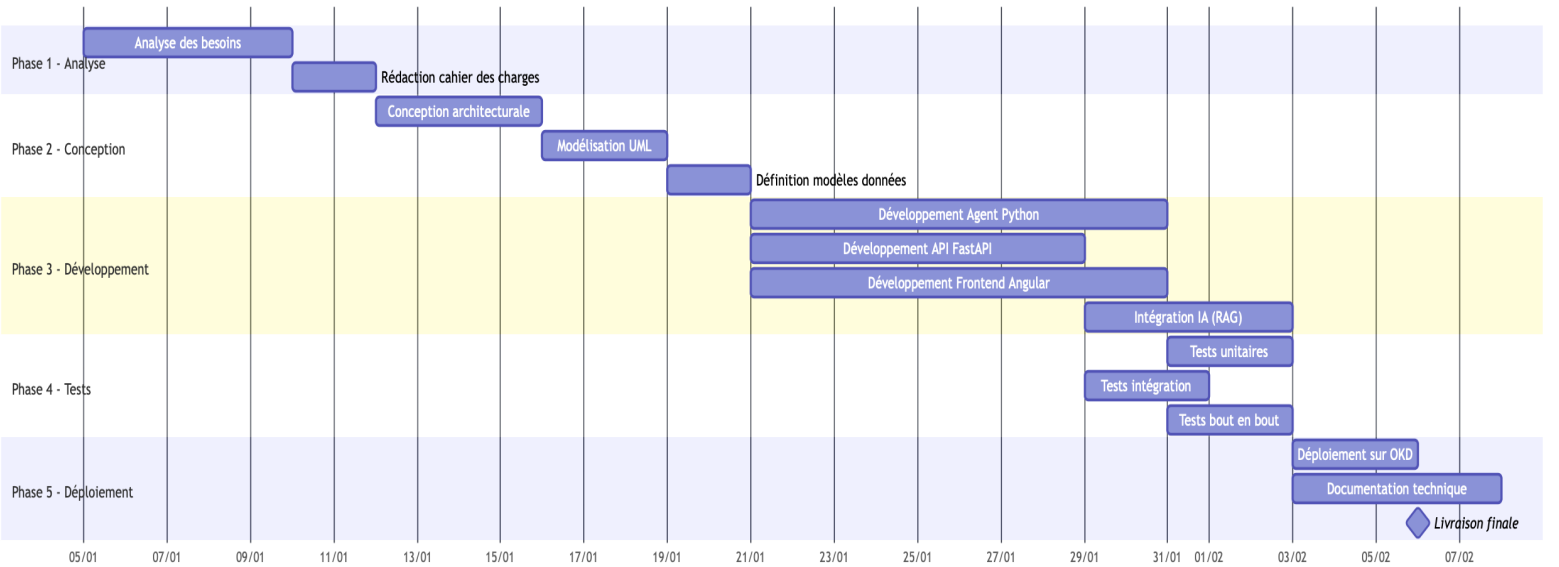


FIGURE 2.3 – Diagramme de Gantt du projet NovaGuard

2.7.3 Calendrier prévisionnel

Cette sous-section synthétise le déroulement du projet sur la période planifiée, en reliant chaque phase aux livrables attendus. Elle permet de suivre l'avancement et de vérifier la cohérence entre les objectifs définis et les échéances prévues.

Phase	Période	Livrables
Analyse des besoins	Semaine 1	Cahier des charges, spécifications fonctionnelles
Conception	Semaines 2-3	Architecture technique, modèles de données, maquettes UI
Développement	Semaines 4-7	Agent Python v1.0, API FastAPI, Frontend Angular, Intégration IA
Tests	Semaines 8-9	Plans de test, rapports de tests, correctifs
Déploiement	Semaine 9	Documentation technique, guide utilisateur, livraison finale

TABLE 2.3 – Calendrier prévisionnel et livrables

Conclusion

Ce chapitre a permis de formaliser les besoins fonctionnels et non fonctionnels de NovaGuard, de modéliser les interactions via un diagramme de cas d'utilisation, de poser les fondations architecturales de la solution et d'établir une planification précise du projet avec un diagramme de Gantt. Le chapitre suivant approfondit l'étude théorique des technologies et concepts mis en œuvre.

Chapitre 3 : Étude théorique

Sommaire du chapitre

3.1 Web Application Firewall (WAF)	21
3.2 Technologies backend	26
3.3 Technologies frontend	28
3.4 Intelligence Artificielle appliquée à la sécurité	29
3.5 Sécurité des communications	30

CHAPITRE 3

ÉTUDE THÉORIQUE

Introduction

Ce chapitre présente les concepts théoriques et les technologies sur lesquels on a basé la solution NovaGuard, afin de justifier les choix techniques effectués. On commence par les fondements des WAF et leur analyse critique, puis on détaille les technologies backend, frontend, IA et sécurité qui constituent l'ossature de la plateforme.

Avant de détailler les technologies utilisées dans NovaGuard, il est nécessaire de présenter le concept central sur lequel repose le projet. La section suivante introduit donc les Web Application Firewalls, leur rôle, leurs modes de déploiement ainsi que leurs limites.

3.1 Web Application Firewall (WAF)

Un Web Application Firewall (WAF) est un dispositif de sécurité applicative placé entre le client et le serveur web, chargé d'inspecter le trafic HTTP/HTTPS entrant et sortant. Contrairement aux firewalls réseau traditionnels qui opèrent aux couches 3 et 4 du modèle OSI, un WAF intervient à la couche 7 (couche applicative), ce qui lui permet d'analyser le contenu des requêtes et des réponses HTTP, d'identifier des patterns d'attaques connus et de bloquer les requêtes malveillantes avant qu'elles n'atteignent l'application [11].

3.1.1 Modes de déploiement

Un WAF peut être déployé selon trois modes principaux, présentés dans le tableau 3.1.

TABLE 3.1 – Modes de déploiement d’un WAF

Mode	Description	Avantages / Inconvénients
Inline (proxy)	Tout le trafic transite par le WAF avant d’atteindre le serveur	Blocage en temps réel / Point de défaillance unique
Out-of-band	Le WAF reçoit une copie du trafic (mirroring)	Pas d’impact sur la latence / Détection uniquement
Module embarqué	Le WAF est intégré directement au serveur web (ex. ModSecurity + Apache)	Faible latence / Couplage fort au serveur

Dans NovaGuard, ModSecurity est déployé en **mode module embarqué** sur Apache, offrant une intégration native et une faible latence de traitement. Dans cette partie, on s’appuie principalement sur le *ModSecurity Handbook* ainsi que sur la documentation OWASP Top 10 et OWASP CRS pour justifier les attaques couvertes, le rôle des règles et les limites d’un WAF [2, 11, 1].

3.1.2 ModSecurity

ModSecurity est un WAF open-source initialement développé comme module Apache, aujourd’hui disponible également pour Nginx et IIS [2]. Il opère selon un moteur de règles basé sur les directives **SecRule**, permettant de définir des conditions de détection et des actions associées. Le Core Rule Set (CRS) d’OWASP fournit un ensemble de règles préconfiguré couvrant les attaques OWASP Top 10 [1].

ModSecurity supporte deux modes de fonctionnement principaux :

- **DetectionOnly** : journalise les requêtes suspectes sans les bloquer, utile en phase d’audit et de calibrage des règles.
- **On** : détection et blocage actifs des requêtes malveillantes en production.

Format du journal d'audit ModSecurity

ModSecurity génère un fichier de journal structuré, `modsec_audit.log`, découpé en sections identifiées par des lettres, comme l'illustre le tableau 3.2.

TABLE 3.2 – Sections du journal d'audit ModSecurity

Section	Contenu
A	En-tête de la transaction : timestamp, identifiant unique, IP source/destination
B	En-têtes HTTP de la requête (méthode, URI, headers)
C	Corps de la requête (POST body, si activé)
F	En-têtes HTTP de la réponse
H	Informations de traitement : règles déclenchées, score d'anomalie, action
Z	Marqueur de fin de transaction

Dans NovaGuard, l'agent parse les sections **A**, **B** et **H** pour extraire les informations essentielles : horodatage, IP source, URI ciblée, règles déclenchées et sévérité. Ces données sont normalisées en JSON avant envoi à l'API, puis affichées dans la page *Logs* du frontend.

3.1.3 Principales attaques couvertes

Les attaques détectées par ModSecurity / CRS incluent notamment [1] :

- **Injection SQL (SQLi)** : insertion de commandes SQL dans les champs de saisie pour manipuler la base de données.
- **Cross-Site Scripting (XSS)** : injection de scripts malveillants exécutés côté client.
- **Local / Remote File Inclusion (LFI/RFI)** : inclusion de fichiers locaux ou distants pour exécution de code arbitraire.
- **Traversée de répertoire** (*path traversal*) : accès non autorisé à des fichiers système via des séquences `../`.
- **Déni de service applicatif (DoS)** : saturation de ressources applicatives.

3.1.4 Limites des WAF traditionnels

Malgré leur utilité, les WAF présentent des limites structurelles importantes qu'il convient d'identifier pour justifier l'apport de NovaGuard [2, 11] :

- **Taux élevé de faux positifs** : les règles statiques bloquent parfois des requêtes légitimes, dégradant l'expérience utilisateur et nécessitant une maintenance continue des règles.
- **Absence de contexte** : un WAF analyse chaque requête de manière isolée, sans corrélation temporelle ni connaissance du comportement global de l'utilisateur.
- **Difficulté de gestion distribuée** : dans une infrastructure multi-serveurs, la cohérence des configurations entre instances n'est pas garantie sans outil centralisé, comme c'est le cas chez Sopra HR.
- **Opacité des logs** : le format brut de `modsec_audit.log` est difficile à exploiter sans outil de parsing et de visualisation dédié.
- **Pas d'adaptation intelligente** : les règles statiques ne s'adaptent pas automatiquement aux nouvelles menaces sans intervention humaine.

C'est précisément pour répondre à ces limites qu'on a conçu NovaGuard comme une couche de gouvernance et d'intelligence au-dessus de ModSecurity, sans le remplacer.

3.1.5 Comparaison des solutions WAF

Plusieurs solutions WAF sont disponibles sur le marché. Le tableau 3.3 présente une analyse comparative des principales alternatives évaluées avant le choix de ModSecurity pour NovaGuard.

TABLE 3.3 – Comparaison des principales solutions WAF

Critère	ModSecurity	AWS WAF	Cloudflare WAF
Type	Open-source, embarqué	Managé, cloud	Managé, cloud
Personnalisation	Très élevée (règles SecRule)	Modérée (règles AWS)	Limitée
Intégration OWASP CRS	Native	Partielle	Partielle
Coût	Gratuit (open-source)	Payant (à l'usage)	Payant (abonnement)
Dépendance cloud	Aucune (on-premise)	Forte (AWS)	Forte (Cloudflare)
Gestion des logs	Fichier local (mod-sec_audit.log)	CloudWatch / S3	Dashboard Cloudflare
Scalabilité	Manuelle	Automatique	Automatique
Adapté à on-premise	Oui	Non	Non

Le choix de **ModSecurity** dans NovaGuard repose sur plusieurs raisons déterminantes :

- Sa **flexibilité** : les règles **SecRule** permettent une personnalisation fine, indispensable pour les environnements Sopra HR.
- Son **intégration native avec OWASP CRS** : la couverture des attaques est étendue et régulièrement mise à jour par la communauté.
- Son **caractère open-source** : aucun coût de licence, contrôle total sur la configuration.
- Sa **compatibilité on-premise** : indispensable dans le contexte d'infrastructure hébergée de Sopra HR, sans dépendance à un fournisseur cloud.

L'étude des WAF et de ModSecurity a permis de comprendre la base sécuritaire sur laquelle s'appuie NovaGuard. Il convient maintenant de présenter les technologies backend choisies pour centraliser, valider et persister les données collectées.

3.2 Technologies backend

Dans cette section, on présente les technologies backend retenues pour NovaGuard ainsi que leur rôle dans l’architecture de la solution.

3.2.1 FastAPI

FastAPI est un framework web Python moderne et asynchrone, publié en 2018 par Sebastián Ramírez [3]. La documentation officielle FastAPI constitue ici la référence principale pour justifier le choix du framework, la validation des schémas, l’exposition des endpoints REST et la génération automatique de la documentation Swagger/OpenAPI. Il repose sur deux bibliothèques fondamentales : **Starlette** pour la gestion des requêtes HTTP asynchrones (moteur ASGI) et **Pydantic** pour la validation et la sérialisation des données. FastAPI génère automatiquement une documentation interactive Swagger UI et ReDoc à partir des annotations de types Python, conformément aux standards OpenAPI 3.0 et JSON Schema.

Architecture interne de FastAPI

L’architecture interne de FastAPI dans NovaGuard suit un découpage en couches clairement séparées, illustré dans la figure 3.1 :

1. **Couche routes (Routers)** : définit les endpoints REST et délègue le traitement aux services. Chaque ressource (sources, règles, événements, analyse IA) possède son propre router Angular.
2. **Couche validation (Pydantic Schemas)** : valide et désérialise automatiquement les données entrantes. Toute donnée non conforme au schéma est rejetée avec une réponse HTTP 422.
3. **Couche service (Business Logic)** : contient la logique métier : traitement du payload de l’agent, appel au module IA, génération de règles custom.
4. **Couche persistance (SQLAlchemy ORM)** : abstraction de la base de données PostgreSQL. Les modèles SQLAlchemy mappent les entités métier aux tables relationnelles.
5. **Couche sécurité (Middleware)** : vérification du token JWT Keycloak sur chaque requête protégée via un middleware d’authentification.

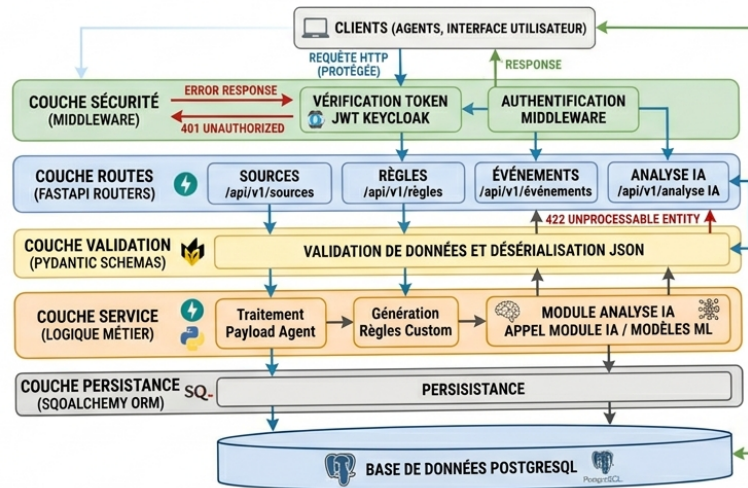


FIGURE 3.1 – Architecture en couches de FastAPI dans NovaGuard

Gestion asynchrone et performances

FastAPI exploite `async/await` de Python pour traiter les requêtes de manière non bloquante. Cela est particulièrement important dans NovaGuard, où plusieurs agents peuvent envoyer simultanément des payloads de collecte à l'API. Le moteur ASGI (Uvicorn) gère la concurrence sans créer un thread par requête, réduisant ainsi la consommation mémoire et améliorant le débit.

Validation des données avec Pydantic

Chaque payload entrant est validé par un schéma Pydantic avant tout traitement. Par exemple, le payload de l'agent est validé contre un modèle `WafCollectPayload` qui vérifie :

- La présence des champs obligatoires (`agent_id`, `hostname`, `collected_at`).
- Le format ISO 8601 du timestamp.
- La structure imbriquée des objets `modsecurity` et `events`.

Toute non-conformité génère automatiquement une réponse HTTP 422 avec le détail des erreurs de validation, sans qu'aucun code de gestion d'erreur supplémentaire ne soit nécessaire.

3.2.2 PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle open-source, reconnu pour sa robustesse, sa conformité aux standards SQL et ses capacités avancées (types JSON natifs, indexation partielle, transactions ACID) [12]. Dans NovaGuard, PostgreSQL stocke l'ensemble des données collectées par les agents : sources WAF, règles actives, fichiers de configuration, événements de sécurité et règles custom générées par IA. La documentation officielle PostgreSQL est utilisée dans cette section pour justifier ce choix, notamment l'usage du type `JSONB` pour stocker les payloads WAF de manière flexible et interrogeable.

On a choisi PostgreSQL pour sa capacité à stocker nativement des colonnes de type `JSONB`, ce qui permet de persister les payloads de l'agent sans perte d'information tout en conservant la possibilité d'effectuer des requêtes JSON complexes.

3.2.3 OKD – Origin Kubernetes Distribution

OKD est la version communautaire d'OpenShift, construite sur Kubernetes et orientée plateforme cloud-native [5, 7, 6]. Les documentations officielles d'OKD, de Kubernetes et d'OpenShift servent ici de base pour expliquer le déploiement retenu, l'organisation en pods et services ainsi que les mécanismes de disponibilité de la plateforme. Elle fournit les primitives d'orchestration (pods, services, déploiements), l'autoscaling, la gestion des configurations et secrets, ainsi que des capacités natives de rolling update et de résilience applicative [6]. Dans NovaGuard, on a déployé l'API FastAPI sur OKD avec un endpoint WAF exposé en HTTPS, afin de garantir la disponibilité du service et la sécurisation des échanges entre les composants [7, 8].

Après avoir présenté les technologies utilisées côté serveur pour traiter les données, il est nécessaire d'aborder la partie visible de la solution. La section suivante décrit donc les choix frontend permettant de restituer ces informations de manière claire et exploitable.

3.3 Technologies frontend

Dans cette section, on présente les principales technologies frontend retenues pour concevoir une interface utilisateur moderne, réactive et adaptée aux besoins de supervision de la plateforme NovaGuard AI.

3.3.1 Angular 17

Angular 17 introduit la compilation standalone (sans NgModule), améliorant les performances de démarrage de l'application [4]. Il offre un framework complet pour la construction d'interfaces web réactives, avec injection de dépendances, routing avancé, guards d'authentification et gestion d'état via RxJS. On a utilisé Angular 17 pour le frontend de NovaGuard pour les raisons suivantes :

- **Architecture standalone** : les composants sont auto-suffisants, ce qui simplifie la maintenance et les tests unitaires.
- **Guards Keycloak** : les routes sensibles sont protégées par des guards Angular qui vérifient la validité du token JWT avant d'autoriser la navigation.
- **HttpClient interceptors** : l'ajout automatique du Bearer token sur toutes les requêtes API est géré par un intercepteur centralisé.
- **RxJS Observables** : la gestion des flux de données asynchrones (polling des logs, mise à jour des tableaux de bord) est naturellement supportée.

L'interface utilisateur assure la visualisation des données collectées, mais NovaGuard vise également à assister l'analyse de sécurité. C'est dans cette logique que la section suivante présente l'apport de l'intelligence artificielle dans l'interprétation des événements et la génération de recommandations.

3.4 Intelligence Artificielle appliquée à la sécurité

3.4.1 Retrieval-Augmented Generation (RAG)

Le RAG est une approche combinant la récupération d'informations pertinentes dans une base de connaissances (*retrieval*) avec la génération de texte par un grand modèle de langage (*generation*) [10]. L'article de Lewis et al. (2020) est mobilisé ici comme fondement scientifique pour justifier l'intégration d'un fallback RAG local au sein de NovaGuard. Dans NovaGuard, on a intégré un RAG local comme fallback lorsque le service IA externe est indisponible. Ce RAG permet de générer des recommandations de sécurité et des règles ModSecurity custom basées sur les données collectées par l'agent.

3.4.2 Exécution locale avec Ollama (Mistral)

Afin de renforcer l'autonomie de la plateforme, on a intégré **Ollama** comme moteur d'exécution locale des modèles LLM [14]. Cette approche permet de lancer **Mistral** [15] localement sur le serveur d'analyse, sans dépendre d'une API externe pour les cas critiques ou les environnements isolés.

Dans NovaGuard, Ollama est utilisé comme couche d'inférence locale pour :

- générer des recommandations de sécurité même en cas d'indisponibilité du service IA distant,
- proposer des règles ModSecurity custom adaptées au contexte observé,
- réduire la latence et améliorer la confidentialité des données sensibles.

Cette architecture hybride (service IA externe + Ollama/Mistral local) [14, 15] améliore la résilience globale du module IA et garantit la continuité opérationnelle.

3.4.3 Détection d'anomalies

Les modèles d'apprentissage non supervisé (clustering K-means, Isolation Forest) permettent d'identifier des comportements inhabituels dans les logs d'événements WAF, sans nécessité de données labellisées. Ces techniques constituent une base pour l'évolution future de NovaGuard.

L'intégration de l'intelligence artificielle enrichit les capacités d'analyse de la plateforme, mais elle doit s'inscrire dans un environnement sécurisé. La section suivante présente donc les mécanismes retenus pour protéger les communications et contrôler les accès.

3.5 Sécurité des communications

Cette section présente les mécanismes retenus pour sécuriser les échanges entre les différents composants de NovaGuard ainsi que l'accès aux ressources de la plateforme.

3.5.1 Keycloak

Keycloak est un serveur d'identité et d'accès open-source (IAM) supportant les protocoles OAuth 2.0 et OpenID Connect [9]. La documentation officielle Keycloak est utilisée dans cette partie pour justifier l'usage du SSO, des flux OAuth 2.0 / OpenID Connect et des tokens JWT dans la sécurisation des accès. Dans NovaGuard, il assure l'authentification SSO des utilisateurs du frontend et délivre des tokens JWT pour sécuriser les appels API.

3.5.2 TLS et vérification SSL

Toutes les communications entre l’agent, l’API et le frontend sont chiffrées via TLS. L’agent dispose d’un paramètre `verify_ssl` permettant de désactiver la vérification en environnement de test.

Conclusion

Ce chapitre a permis de présenter les fondements théoriques de la solution NovaGuard, depuis l’analyse critique des WAF et le choix justifié de ModSecurity, jusqu’aux technologies backend (FastAPI, PostgreSQL, OKD), frontend (Angular 17) et IA (RAG, Ollama/Mistral). Ces bases théoriques guident directement les choix d’implémentation détaillés dans le chapitre suivant.

CHAPITRE 4

RÉALISATION

Introduction

Ce chapitre présente en détail l'implémentation des composants principaux de NovaGuard que on a développés : l'agent de collecte déployé sur RHEL, l'API WAF backend, le frontend Angular et le module d'intelligence artificielle. Pour chaque composant, on décrit la démarche suivie, les choix techniques retenus, la configuration réelle utilisée, les difficultés rencontrées et les résultats obtenus.

Après avoir présenté les fondements théoriques et les choix technologiques de la solution, il convient d'expliquer comment ceux-ci ont été mis en œuvre concrètement.

4.1 Réalisation par phase

Cette section décrit le déroulement du projet selon une logique progressive, de l'analyse initiale jusqu'à la mise en production.

4.1.1 Phase 1 : Analyse des besoins

La réalisation a commencé par un cadrage fonctionnel avec l'équipe ProdOps et les parties prenantes sécurité. On a identifié les limites de l'existant : collecte manuelle, absence de vue centralisée, faible traçabilité et absence de supervision des logs d'audit ModSecurity. On a formalisé les besoins prio-

ritaires : centralisation des données WAF, consultation rapide par serveur, supervision de `modsec_audit.log` et assistance à l'analyse par IA. Cette phase a permis d'établir un périmètre réaliste et des critères d'acceptation mesurables.

4.1.2 Phase 2 : Conception

Sur la base des besoins validés, on a conçu une architecture en trois couches : agent, API centrale et interface web. On a défini les flux de données, le schéma du payload JSON, les modèles de persistance et les endpoints REST nécessaires. Cette phase a également fixé les choix techniques (Python / FastAPI / Angular / OKD) et les exigences de sécurité (authentification Keycloak, chiffrement TLS, contrôle d'accès API).

4.1.3 Phase 3 : Développement

La phase de développement a été organisée en trois volets complémentaires : l'agent de collecte, le dashboard de supervision et le module d'analyse IA, développés en parallèle pour optimiser les délais.

Développement de l'agent

On a développé l'agent Python chargé d'extraire l'état de ModSecurity (fichiers de configuration, règles actives et événements d'audit), de normaliser les données et de les envoyer vers l'API centrale. La supervision des logs `modsec_audit.log` a été activée dans un second temps, sur demande de l'encadrant.

```
avril 04 19:06:42 modsec-blocking.local python3[4220]: "collected_at": datetime.utcnow().isoformat() + "Z",
avril 04 19:06:42 modsec-blocking.local python3[4220]: 2026-04-04 19:06:42,567 [INFO] [✓] Envoi réussi (1 events)
avril 04 19:07:12 modsec-blocking.local python3[4220]: 2026-04-04 19:07:12,894 [INFO] Fichiers trouvés: /var/log/modsec_audit.log
avril 04 19:07:12 modsec-blocking.local python3[4220]: 2026-04-04 19:07:12,894 [INFO] Lecture /var/log/modsec_audit.log (0 bytes)
avril 04 19:07:12 modsec-blocking.local python3[4220]: 2026-04-04 19:07:12,895 [INFO] Aucun nouvel événement ModSecurity (vérifier log_paths et permissions)
avril 04 19:07:12 modsec-blocking.local python3[4220]: /opt/novaguard_agent/novaguard_modsec_agent.py:458: DeprecationWarning: datetime.datetime.utcnow() is
deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
avril 04 19:07:12 modsec-blocking.local python3[4220]: "collected_at": datetime.utcnow().isoformat() + "Z",
```

FIGURE 4.1 – Journalisation de l'agent NovaGuard sur RHEL

Développement du dashboard

On a réalisé le dashboard Angular pour offrir une visualisation centralisée de la posture WAF : inventaire des serveurs, état des règles, consultation des configurations, suivi des événements et affichage des logs d'audit ModSecurity.

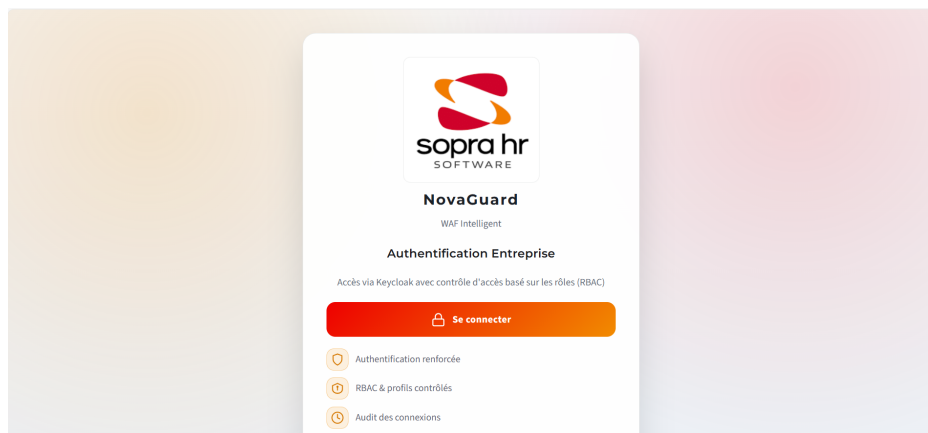


FIGURE 4.2 – Dashboard de gouvernance WAF – vue générale

Intégration du module IA

On a intégré le module IA pour assister l'analyse de sécurité et proposer des recommandations actionnables via une architecture hybride (service distant + RAG local + Ollama/Mistral).

4.1.4 Phase 4 : Tests et validation

On a exécuté des tests à trois niveaux : unitaires (par composant), intégration (entre composants) et bout en bout (flux complet agent → API → frontend). Les vérifications ont couvert la conformité du payload, la robustesse de la collecte, la fiabilité des endpoints, la persistance en base et le comportement de l'interface sous différentes conditions d'erreur.

4.1.5 Phase 5 : Déploiement et documentation

La dernière phase a consisté à déployer les composants sur l'environnement cible (OKD pour l'API et le frontend, serveurs WAF RHEL pour l'agent) et à rédiger la documentation technique et opérationnelle de la solution.

La présentation des phases de réalisation a permis de situer l'évolution globale du projet, depuis l'analyse jusqu'au déploiement. Il est maintenant nécessaire d'étudier plus précisément le premier composant technique de la solution : l'agent de collecte déployé sur les serveurs RHEL.

4.2 Agent NovaGuard sur RHEL

Dans cette section, on présente l’agent NovaGuard déployé sur RHEL, son rôle dans la collecte des données WAF ainsi que son intégration sécurisée avec l’API centralisée de la plateforme.

4.2.1 Rôle de l’agent

L’agent Python constitue le composant de collecte déployé directement sur chaque serveur RHEL hébergeant ModSecurity. Il a été conçu pour :

- détecter et lire les fichiers de configuration ModSecurity,
- auditer les règles actives et compter les fichiers chargés,
- parser les transactions d’audit du fichier `modsec_audit.log`,
- enrichir les événements (sévérité, règle déclenchée, action, IP source, URI),
- envoyer les résultats normalisés à l’API centrale via un payload JSON,
- conserver un état local (`state.json`) pour la lecture incrémentale.

4.2.2 Architecture interne de l’agent

Le tableau 4.1 présente l’organisation fonctionnelle des modules implémentés dans l’agent.

TABLE 4.1 – Composants internes de l’agent NovaGuard

Module	Rôle
CLI & configuration	Parsing des arguments CLI et chargement de <code>config.json</code>
Détection ModSecurity	Détection de présence, version et chemins de base sur RHEL
Collecte configurations	Lecture des fichiers <code>.conf</code> avec métadonnées (taille, mtime, contenu)
Audit règles	Scan des répertoires de règles, résolution des symlinks et comptage des fichiers chargés
Collecte événements	Parsing incrémental de <code>modsec_audit.log</code> (sections A/B/H)
Normalisation	Conversion des dates en ISO 8601, gestion des valeurs <code>null</code> , troncature des configs volumineuses
Client HTTP	POST JSON vers l’API, gestion SSL, retries et timeouts
État local	Lecture incrémentale sans doublon via <code>state.json</code> (offset par fichier)

4.2.3 Structure fonctionnelle implémentée

Les fonctions de l’agent sont organisées en groupes cohérents :

- **Initialisation** : `load_config`, `setup_logger` — chargement de la configuration JSON et mise en place de la journalisation structurée avec niveaux DEBUG / INFO / WARNING / ERROR.
- **État incrémental** : `read_state`, `write_state` — persistance de la position de lecture dans les fichiers de logs afin d’éviter les doublons entre deux exécutions.
- **Découverte et lecture** : `discover_files`, `read_new_content` — identification des fichiers de logs et lecture des nouvelles entrées depuis le dernier offset enregistré.
- **Parsing des logs** : `parse_json_lines`, `split_transactions`, `parse_transaction`, `parse_modsec` — découpage et interprétation des transactions ModSecurity aux formats JSON et texte natif.
- **Détection et audit** : `detect_modsecurity_version`, `audit_rules` — identification de la version installée et comptage des règles actives

dans les répertoires configurés.

- **Collecte complète** : `collect_modsecurity_audit_log`, `collect_modsecurity_config`, `detect_modsecurity` — orchestration de l'ensemble de la collecte WAF.
- **Payload et envoi** : `build_payload`, `send_payload`, `save_payload` — construction du payload JSON normalisé, envoi vers l'API et sauvegarde locale de secours.
- **Boucle principale** : `collect_once`, `run` — orchestration globale en mode ponctuel (`-run-once`) ou continu.

4.2.4 Supervision des logs ModSecurity

À la demande de l'encadrant, on a étendu les capacités de l'agent pour inclure la supervision du fichier `modsec_audit.log` généré par ModSecurity. Cette extension constitue un apport fonctionnel majeur : les logs d'audit contiennent les transactions HTTP ayant déclenché des règles de sécurité, et leur exploitation permet d'identifier les menaces actives, les faux positifs et les comportements anormaux.

Activation de la collecte des événements

La supervision des logs s'active en positionnant le paramètre `collect_events` à `true` dans `config.json`. Le tableau 4.2 résume les paramètres concernés.

TABLE 4.2 – Paramètres de configuration liés à la supervision des logs

Paramètre	Valeur	Rôle
<code>collect_events</code>	<code>true</code>	Active le parsing de <code>modsec_audit.log</code>
<code>audit_log_paths</code>	<code>["/var/log/modsec_audit.log"]</code>	Chemin(s) du ou des fichiers d'audit à superviser
<code>max_events</code>	100	Nombre maximum d'événements envoyés par collecte
<code>state_file</code>	<code>state.json</code>	Fichier de persistance de l'offset de lecture

Logique de parsing du fichier d'audit

Le fichier `modsec_audit.log` est structuré en transactions délimitées par des marqueurs. Chaque transaction est découpée en sections identifiées par une lettre. L'agent extrait les informations des sections suivantes :

- **Section A** : horodatage, identifiant unique de transaction (`unique_id`), IP source et port de destination.
- **Section B** : méthode HTTP, URI ciblée, version du protocole et en-têtes de la requête (`Host`, `User-Agent`).
- **Section H** : règles ModSecurity déclenchées (`id`, `msg`, `severity`), score d'anomalie total et action appliquée (`pass`, `block`, `redirect`).

La lecture est incrémentale : l'agent mémorise l'offset dans `state.json` et reprend à cet offset lors de l'exécution suivante, évitant le retraitement des entrées déjà envoyées.

Structure d'un événement normalisé

Après parsing, chaque transaction est normalisée en un objet JSON intégré dans le tableau `events` du payload :

```
Action: Intercepted (phase 2)
Stopwatch: 1775326005981320 11723 (- - -)
Stopwatch2: 1775326005981320 11723; combined=8514, p1=3857, p2=3691, p3=0, p4=0, p5=966, sr=0, sw=0, l=0, gc=0
Response-Body-Transformed: Dechunked
Producer: ModSecurity for Apache/2.9.12 (http://www.modsecurity.org/); OWASP_CRS/4.0.0.
Server: Apache/2.4.63 (Red Hat Enterprise Linux)
Engine-Mode: "ENABLED"

--b0dfbf54-Z--
```

FIGURE 4.3 – Événement normalisé extrait de `modsec_audit.log`

4.2.5 Normalisation des données

Avant l'envoi, l'agent applique les normalisations suivantes :

- **Dates** : conversion systématique en format ISO 8601.
- **Valeurs manquantes** : `null` si les données ne sont pas disponibles.
- **Configs volumineuses** : troncature du contenu si la taille dépasse le seuil configuré.
- **Logs** : sections A/B/H transformées en champs lisibles et typés.
- **Score d'anomalie** : converti en entier, `null` si absent.

4.2.6 Configuration réelle utilisée

Cette sous-section présente la configuration réellement utilisée pour exécuter l'agent NovaGuard, avec les paramètres nécessaires à la collecte et à la transmission des données.

Plus précisément, le fichier `config.json` définit un agent identifié par `novaguard-agent-001`, exécuté toutes les 30 secondes, avec envoi des données vers une API HTTPS et vérification SSL activée. La configuration prévoit également jusqu'à trois réessais espacés de 10 secondes en cas d'échec, un stockage local dans `./data`, un fichier d'état `./state.json` et un niveau de journalisation `INFO`. Dans cette version de configuration, l'option `collect_events` est désactivée, ce qui limite l'exécution au mode batch sans collecte temps réel des événements.

Point critique : activation de la collecte des événements

Dans la configuration initiale, `collect_events` était positionné à `false`, limitant l'agent à la collecte des métadonnées de configuration et de règles. Sur demande de l'encadrant, on a activé ce paramètre (`true`) pour étendre la supervision aux transactions d'audit de `modsec_audit.log`. Cette modification alimente la page *Logs* du frontend et offre une traçabilité complète des événements de sécurité.

4.2.7 Robustesse et gestion d'erreurs

- **Fichiers manquants** : ignorés avec avertissement, collecte partielle envoyée.
- **Permissions insuffisantes** : erreurs capturées, l'exécution continue.
- **Erreurs HTTP** : retries configurés (`max_retries=3`), timeouts ajustables.
- **Mode config-only** : si `collect_events=false`, le tableau `events` est vide mais le payload est quand même envoyé.
- **État local** : `state.json` garantit la lecture incrémentale sans doublon.
- **Logs malformés** : transactions non conformes ignorées avec avertissement, sans interruption du traitement des entrées suivantes.

4.2.8 Difficultés rencontrées côté agent

Le développement de l'agent sur RHEL a soulevé plusieurs défis techniques :

- **Permissions sur les fichiers ModSecurity** : sur RHEL, les fichiers de configuration et les logs d’audit sont protégés par des droits restrictifs (`root:root`, mode 640). On a dû configurer un accès en lecture pour l’utilisateur système dédié à l’agent via un groupe POSIX spécifique, sans élever les privilèges de manière permanente.
- **Format irrégulier de `modsec_audit.log`** : selon la version de ModSecurity (v2 ou v3) et la configuration du serveur, le format du fichier d’audit peut varier (texte natif ou JSON). On a implémenté un par-seur dual-format capable de détecter automatiquement le format et d’adapter le traitement en conséquence.
- **Lecture incrémentale robuste** : lors des premiers tests, l’agent retraissait l’intégralité du fichier de log à chaque exécution, provoquant des doublons en base. On a introduit le mécanisme `state.json` avec persistance de l’offset byte pour résoudre ce problème.
- **Rotation des logs** : ModSecurity effectue une rotation périodique du fichier d’audit (`logrotate`). On a ajouté une détection de rotation basée sur la comparaison de l’inode du fichier entre deux exécutions, déclenchant une réinitialisation de l’offset en cas de rotation détectée.

4.2.9 Déploiement et planification sur RHEL

Cette sous-section décrit les étapes retenues pour déployer l’agent sur RHEL et planifier son exécution dans un cadre d’exploitation régulier.

Listing 4.1 – Installation et lancement de l’agent NovaGuard sur RHEL

```
# Creation de l'environnement virtuel Python
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Execution unique avec logs verbeux (test)
python3 novaguard_modsec_agent.py -c config.json

# Execution en mode continu (production)
python3 novaguard_modsec_agent.py -c config.json
```

La planification périodique s’effectue via une entrée cron ou un service `systemd`, permettant une collecte automatique toutes les N minutes selon la criticité de l’environnement.

L’agent NovaGuard assure la collecte locale des configurations, des règles et des événements ModSecurity. Ces données doivent ensuite être transmises,

validées et centralisées, ce qui constitue le rôle principal de l'API WAF Backend présentée dans la section suivante.

4.3 API WAF Backend

Dans cette section, on présente l'API backend dédiée à la gestion centralisée des données WAF, en mettant en évidence son architecture, ses principales fonctionnalités et son rôle dans la communication entre les différents composants de NovaGuard AI.

4.3.1 Choix techniques

On a développé l'API backend avec les technologies suivantes :

- **FastAPI** pour les endpoints REST et la documentation Swagger automatique,
- **Pydantic** pour la validation des contrats de données (payload agent),
- **SQLAlchemy** comme ORM pour l'abstraction de la couche PostgreSQL,
- **PostgreSQL** pour la persistance en environnement de production,
- déployée sur **OKD** avec exposition HTTPS sécurisée.

4.3.2 Flux interne de traitement d'un payload

Le flux interne de NovaGuard commence lorsque l'agent ModSecurity (RHEL) collecte les événements de sécurité et envoie un payload JSON à l'API FastAPI (Windows) via l'endpoint de collecte. Le backend valide ce payload avec le schéma Pydantic, stocke les métadonnées utiles (agent, hôte, horodatage, source) ainsi que le JSON complet en base SQLite, puis expose ces données aux endpoints de consultation. Lors de la lecture, une normalisation applicative est appliquée (statut, sévérité, temps UTC) pour alimenter les vues logs, statistiques et timeline du dashboard.

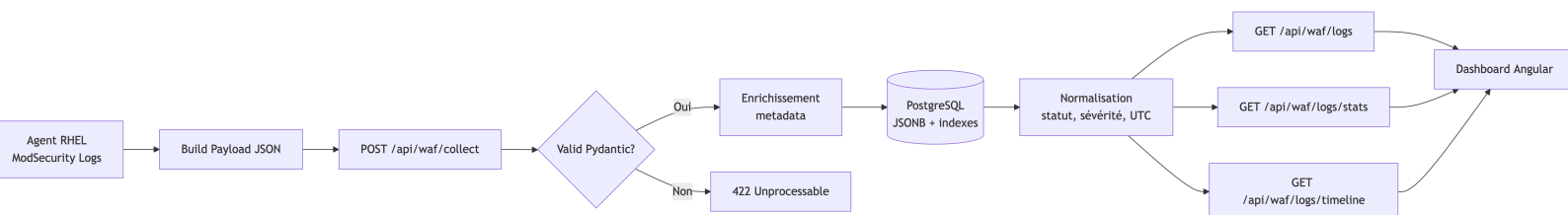


FIGURE 4.4 – Diagramme de séquence – traitement d'un payload par l'API

4.3.3 Modèles de données SQLAlchemy

Les entités persistées en PostgreSQL sont modélisées via SQLAlchemy. Le tableau 4.3 présente les principales tables et leurs colonnes clés.

TABLE 4.3 – Modèles de données SQLAlchemy de NovaGuard

Table	Colonnes principales
waf_sources	id, agent_id, hostname, ip, os, modsec_version, last_seen
waf_config_files	id, source_id (FK), path, size, mtime, content
waf_rules_audit	id, source_id (FK), rules_count, collected_at
waf_events	id, source_id (FK), unique_id, timestamp, client_ip, method, uri, anomaly_score, action, rules_triggered (JSONB)
waf_custom_rules	id, source_id (FK), title, rule_text, mode, ia_report, created_at

4.3.4 Validation du payload avec Pydantic

Chaque payload entrant est validé par le schéma `WafCollectPayload` avant tout traitement. Ce schéma vérifie notamment :

- La présence des champs obligatoires (`agent_id`, `hostname`, `collected_at`).
- Le format ISO 8601 du timestamp (`datetime` Python).
- La structure imbriquée des objets `modsecurity` et `events`.
- Le type et le format de chaque champ d'un événement (IP au format `IPv4Address`, sévérité dans une `Enum` définie).

Toute non-conformité génère automatiquement HTTP 422 avec le détail des erreurs de validation, sans code de gestion d'erreur supplémentaire.

4.3.5 Étapes de développement de l'API

Le développement de l'API a été mené de manière progressive afin de garantir une base technique stable avant l'ajout des fonctionnalités métier et des mécanismes de sécurité.

1. Initialisation du projet FastAPI et structuration des routes métiers.
2. Définition des schémas Pydantic et des modèles SQLAlchemy.

3. Implémentation de l'endpoint `/api/waf/collect`.
4. Développement des endpoints de consultation (sources, règles, événements, logs).
5. Intégration de l'endpoint d'analyse IA `/api/waf/analyze`.
6. Connexion à PostgreSQL et validation de la persistance.
7. Sécurisation JWT/Keycloak et tests d'intégration.

4.3.6 Endpoints REST implémentés

Le tableau suivant résume les principaux endpoints REST implémentés dans l'API ainsi que leur rôle dans le fonctionnement global de NovaGuard AI.

TABLE 4.4 – Endpoints REST de l'API WAF NovaGuard

Méthode	Endpoint	Description
POST	<code>/api/waf/collect</code>	Réception et persistance du payload agent
GET	<code>/api/waf/sources</code>	Liste des serveurs WAF enregistrés
GET	<code>/api/waf/rules</code>	Règles actives par serveur
GET	<code>/api/waf/events</code>	Événements de sécurité récents
GET	<code>/api/waf/logs</code>	Logs d'audit ModSecurity avec filtres et pagination
GET	<code>/api/waf/stats</code>	Statistiques filtrées et timeline 24h
POST	<code>/api/waf/analyze</code>	Analyse IA et génération de règles custom
GET	<code>/api/waf/users</code>	Liste des utilisateurs (administration)

4.3.7 Endpoint de consultation des logs

L'endpoint `GET /api/waf/logs` expose les logs d'audit parsés et persistés en base. Il supporte les paramètres de filtrage suivants :

- `source_id` : filtre par serveur WAF source.
- `severity` : filtre par niveau (`CRITICAL`, `WARNING`, `NOTICE`).
- `action` : filtre par action (`block`, `pass`).
- `from_date` / `to_date` : plage temporelle.
- `limit` / `offset` : pagination des résultats.

4.3.8 Difficultés rencontrées côté API

Au cours de l'implémentation du backend, plusieurs difficultés techniques ont été rencontrées et progressivement résolues pour assurer la robustesse du service.

- **Latence OKD au démarrage** : lors du premier déploiement sur OKD, les pods FastAPI mettaient jusqu'à 45 secondes à démarrer en raison du *cold start*. On a configuré des *readiness probes* HTTP et ajusté les ressources CPU/mémoire allouées pour stabiliser le démarrage.
- **Contraintes JSONB PostgreSQL** : la persistance du champ `rules_triggered` en JSONB a nécessité un typage explicite dans SQLAlchemy et une sérialisation Python→JSON avant insertion.
- **Gestion des doublons** : deux agents envoyant simultanément des données pour le même serveur pouvaient créer des doublons dans `waf_sources`. On a résolu ce problème par une contrainte `UNIQUE` sur `(agent_id, hostname)` et une opération *upsert* (`INSERT ON CONFLICT DO UPDATE`).

L'API backend permet de structurer et d'exposer les données collectées par les agents. Afin de rendre ces informations accessibles et exploitables par les utilisateurs, il est nécessaire de disposer d'une interface de visualisation adaptée, développée ici avec Angular.

4.4 Frontend Angular

Dans cette section, on présente le frontend Angular développé pour NovaGuard AI, en mettant l'accent sur son organisation, ses composants principaux et son apport à la visualisation centralisée des informations de sécurité.

4.4.1 Architecture des composants

Cette sous-section présente l'organisation des composants Angular et la manière dont ils collaborent pour structurer l'interface de supervision.

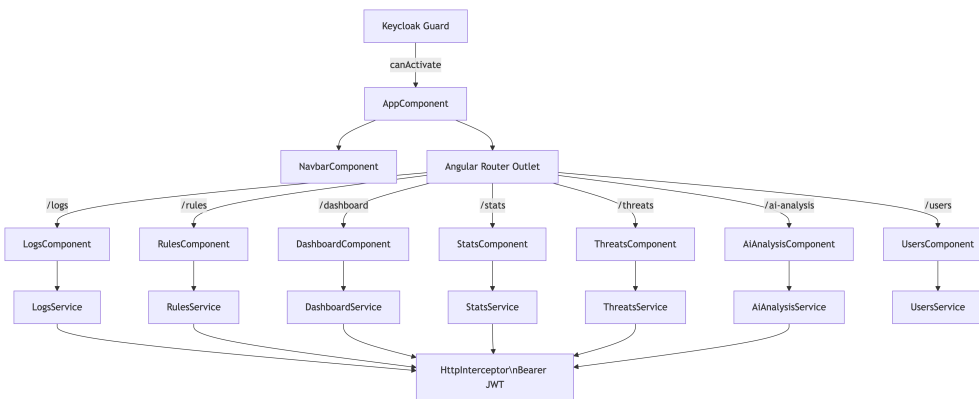


FIGURE 4.5 – Architecture des composants Angular du frontend NovaGuard

4.4.2 Pages et modules réalisés

On a développé le frontend en Angular 17 en mode standalone. L'interface comprend les pages suivantes :

- **Tableau de bord SOC** : vue d'ensemble avec KPI et anneaux statistiques animés.

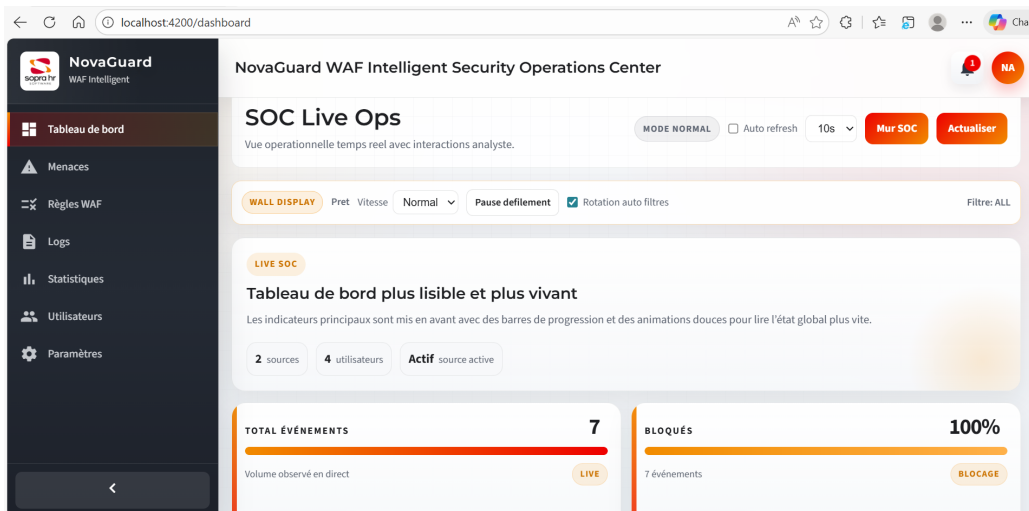


FIGURE 4.6 – Page Dashboard du frontend NovaGuard

- **Menaces** : liste des événements de sécurité avec filtrage par sévérité et par serveur.
- **Logs** : consultation des logs d'audit ModSecurity avec filtres avancés
 - détaillée en section 4.4.3.

Chapitre 4. Réalisation

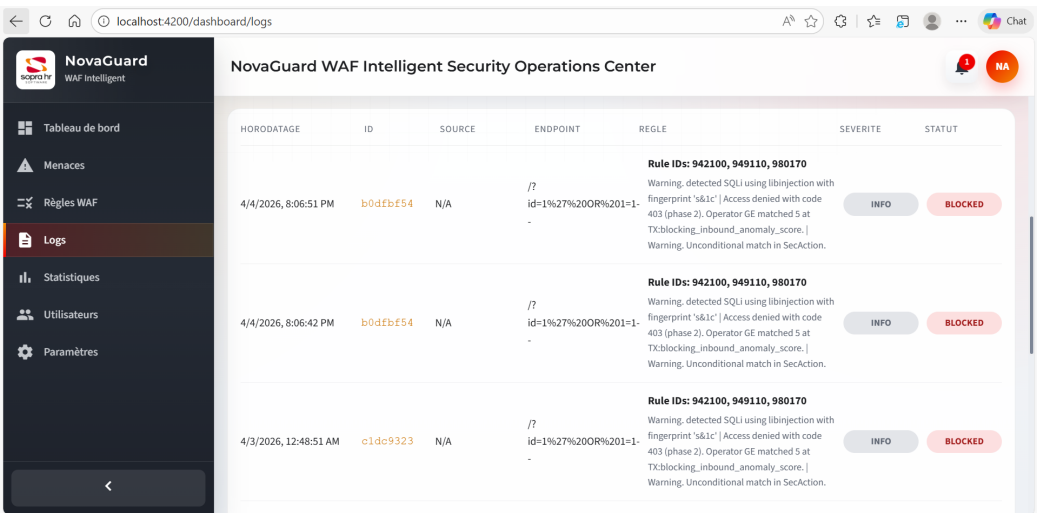


FIGURE 4.7 – Page Logs – consultation des événements d’audit ModSecurity

- **Statistiques analytiques** : graphiques et timeline sur 24 heures.
- **Règles WAF** : visualisation et gestion des règles ModSecurity.

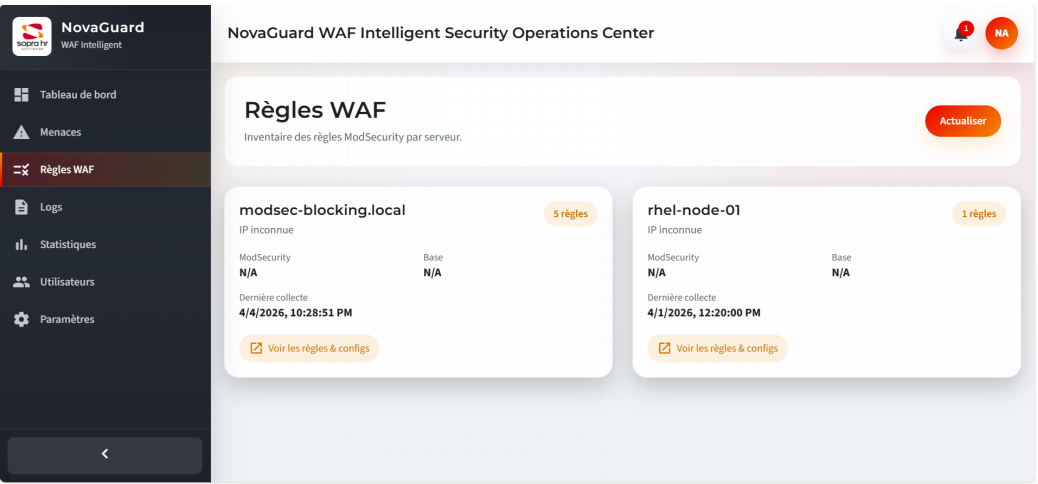


FIGURE 4.8 – Vue frontend de gestion des règles WAF

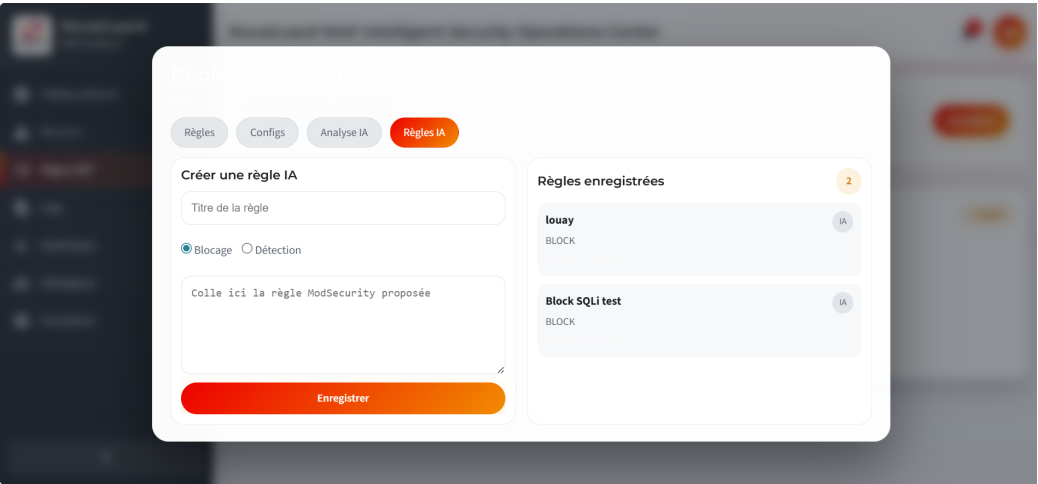


FIGURE 4.9 – Création d’une règle personnalisée depuis le frontend

— **Utilisateurs** : vue d’administration des accès et des rôles.

4.4.3 Page Logs : affichage des événements ModSecurity

La page *Logs* est l’aboutissement de la chaîne de supervision de `modsec_audit.log`. Elle consomme l’endpoint `GET /api/waf/logs` et présente les informations du tableau 4.5.

TABLE 4.5 – Colonnes affichées dans la page Logs du frontend

Colonne	Description
Horodatage	Date et heure de la transaction (format ISO 8601 localisé)
Serveur	Hostname du serveur WAF source
IP source	Adresse IP du client ayant déclenché la règle
Méthode	Méthode HTTP de la requête (GET, POST, PUT...)
URI	URI ciblée par la requête
Règle	Identifiant et message de la règle déclenchée
Sévérité	Niveau de criticité avec badge coloré (rouge = CRITICAL, orange = WARNING, bleu = NOTICE)
Action	Action appliquée par ModSecurity (block / pass)
Score	Score d’anomalie cumulé de la transaction

Les filtres disponibles (serveur, sévérité, action, plage de dates) permettent de restreindre l’affichage. La pagination limite l’affichage à 50 entrées par page pour maintenir les performances.

4.4.4 Difficultés rencontrées côté frontend

Le développement du frontend a également soulevé plusieurs contraintes techniques liées à l’authentification, à l’intégration des données et à l’expérience utilisateur.

- **Intégration Keycloak avec Angular 17 standalone** : la migration vers les composants standalone a rendu certaines bibliothèques Keycloak incompatibles avec l’injection de dépendances traditionnelle. On a adapté la configuration en utilisant `provideKeycloak()` au niveau `bootstrapApplication()`.
- **Gestion des tokens expirés** : lors de sessions longues, les tokens JWT Keycloak expirent et les appels API retournent HTTP 401. On a implémenté un intercepteur Angular qui détecte les 401 et déclenche automatiquement le rafraîchissement du token via Keycloak avant de rejouer la requête.
- **Volume de logs affiché** : lors des premiers tests, la page Logs chargeait l’intégralité des événements sans pagination, provoquant des temps de rendu supérieurs à 3 secondes. On a introduit la pagination côté serveur (`limit/offset`) pour ramener le temps de chargement à moins de 500 ms.

4.4.5 Étapes de développement

La réalisation de l’interface Angular s’est appuyée sur une progression structurée, allant de l’initialisation du projet jusqu’à l’intégration des fonctionnalités avancées.

1. Initialisation du projet Angular 17 et organisation des composants standalone.
2. Création des vues principales (dashboard, menaces, règles, utilisateurs).
3. Intégration des services Angular (`HttpClient`) et de l’intercepteur JWT.
4. Développement de la page Logs avec tableau filtrable et pagination.
5. Ajout des modales de détail des configurations WAF.
6. Développement de la page d’analyse IA et affichage des recommandations.

7. Intégration Keycloak (login, guards de routes, token refresh) et validation UX.

4.4.6 UX et éléments visuels

Une attention particulière a été portée à l'expérience utilisateur afin de rendre la consultation des informations de sécurité plus fluide, lisible et intuitive.

- Anneaux statistiques animés (progression de 0 vers la valeur cible au chargement).
- Badges de sévérité colorés (rouge = CRITICAL, orange = WARNING, bleu = NOTICE).
- Cartes KPI interactives avec indicateurs de statut.
- Timeline d'événements de sécurité sur 24 heures.
- Design responsive adapté à différentes résolutions d'écran.

Le frontend fournit une vision centralisée de l'état des serveurs WAF et des événements de sécurité. Toutefois, l'objectif de NovaGuard est également d'assister l'analyse humaine grâce à des recommandations intelligentes, ce qui justifie l'intégration du module d'intelligence artificielle.

4.5 Module d'Intelligence Artificielle

Dans cette section, on présente le module d'intelligence artificielle intégré à NovaGuard AI, son fonctionnement général ainsi que sa contribution à l'analyse contextuelle et à la génération de recommandations de sécurité.

4.5.1 Approche hybride

L'intégration IA a été conçue en mode hybride pour garantir la continuité de service :

- **Inférence distante** : appel vers un service IA externe en priorité.
- **Fallback local (RAG)** : moteur RAG local basé sur une base de connaissances OWASP CRS + configurations ModSecurity types.
- **Exécution locale via Ollama / Mistral** : modèle de langage open-source exécuté localement pour les environnements isolés.

4.5.2 Cas d'usage implémentés

Le module IA a été appliqué à plusieurs cas d'usage concrets afin d'apporter une assistance directement exploitable par les équipes de sécurité.

- **Aide à l'interprétation des logs** : explication en langage naturel des événements déclenchés par ModSecurity depuis la page Logs.
- **Assistance règles WAF** : proposition de règles **SecRule** personnalisées adaptées au contexte du serveur audité.
- **Recommandations de configuration** : détection des directives mal configurées et propositions de correction.
- **Priorisation analyste** : classement des alertes par niveau de risque.

4.5.3 Exemple de réponse IA intégrée

La figure suivante illustre un exemple de réponse générée par le module IA à partir des données collectées et analysées dans la plateforme.

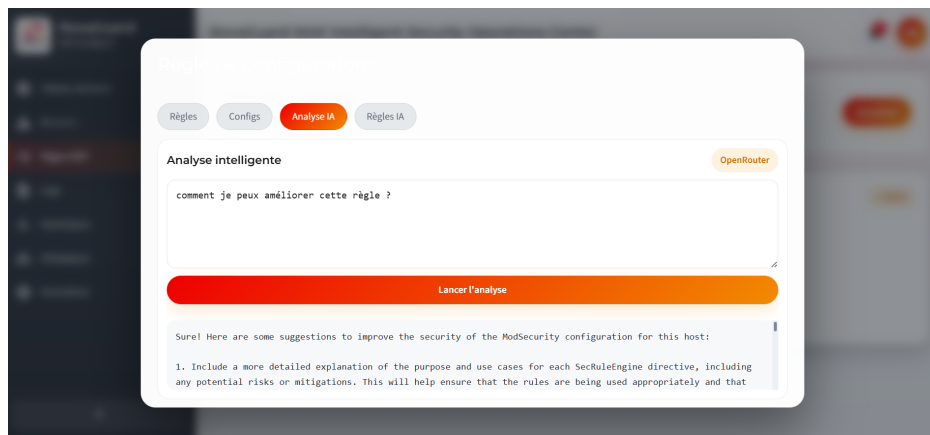


FIGURE 4.10 – Affichage de l'analyse IA dans le frontend NovaGuard

Les recommandations générées portent sur : la politique **SecRuleEngine** par environnement, la réduction des faux positifs par activation progressive, la standardisation de l'organisation des fichiers de configuration, et le suivi des indicateurs de performance après modification des règles.

L'intégration de l'IA enrichit les capacités d'analyse de NovaGuard, mais l'ensemble de la plateforme doit rester sécurisé de bout en bout. La section suivante présente donc les mécanismes mis en place pour protéger les échanges, les accès et les opérations sensibles.

4.6 Sécurité de la solution

Dans cette section, on présente les mécanismes de sécurité mis en place pour protéger les échanges, renforcer le contrôle d'accès et garantir la fiabilité

globale de la plateforme NovaGuard AI.

TABLE 4.6 – Mécanismes de sécurité de la solution NovaGuard

Mécanisme	Implémentation
Authentification	Keycloak (OAuth 2.0 / OpenID Connect), SSO frontend, tokens JWT pour les appels API
Chiffrement	TLS sur toutes les communications agent – API – frontend; <code>verify_ssl=true</code> en production
Contrôle d'accès	RBAC Keycloak côté serveur, guards Angular côté frontend, vérification JWT sur chaque endpoint API
Journalisation	Logs des accès, des opérations sensibles et des erreurs pour traçabilité et audit
Lecture seule agent	L'agent accède aux fichiers ModSecurity en lecture seule, sans aucune écriture sur le système

Conclusion

Ce chapitre a présenté l'implémentation détaillée des quatre composants de NovaGuard. L'agent de collecte, étendu pour superviser `modsec_audit.log`, transmet des payloads enrichis à l'API FastAPI sur OKD, qui les valide, les persiste et les expose au frontend Angular. La page Logs offre une consultation filtrée et paginée des événements d'audit ModSecurity en temps quasi réel. Le chapitre suivant présente une analyse des tests approfondie des résultats obtenus et les perspectives d'évolution de la solution.

Chapitre 5 : Tests et analyse des résultats

Sommaire du chapitre

5.1 Stratégie de tests.....	53
5.2 Résultats obtenus	58

CHAPITRE 5

TESTS ET ANALYSE DES RÉSULTATS

Introduction

Ce chapitre présente la stratégie de tests qu'on a mise en place pour valider la solution NovaGuard, ainsi que l'analyse des résultats obtenus.

Après la réalisation des composants de NovaGuard, il est indispensable de vérifier leur bon fonctionnement de manière structurée. La section suivante présente donc la stratégie de tests adoptée pour valider chaque composant ainsi que le flux global de la solution.

5.1 Stratégie de tests

On a adopté une stratégie de tests à trois niveaux, couvrant chaque composant de manière indépendante avant de valider le flux bout en bout.

5.1.1 Tests de l'agent

Pour valider l'agent, on a réalisé les tests suivants :

TABLE 5.1 – Tests de l’agent NovaGuard

Test	Description	Résultat
Collecte configuration	Vérification de la lecture des fichiers <code>.conf</code> et de leurs méta-données (taille, mtime, contenu)	OK
Audit règles	Comptage correct des règles actives dans le répertoire <code>/etc/modsecurity/rules</code>	OK
Parsing logs	Extraction correcte des sections A/B/H de <code>modsec_audit.log</code> et normalisation JSON	OK
Lecture incrémentale	Vérification que seules les nouvelles entrées sont traitées entre deux exécutions successives (offset <code>state.json</code>)	OK
Robustesse fichiers absents	Simulation d’un fichier de config manquant : l’agent continue sans interruption avec avertissement	OK
Robustesse permissions	Simulation de permissions insuffisantes sur le log d’audit : erreur capturée, collecte partielle envoyée	OK
Envoi HTTP / HTTPS	Vérification du POST vers l’API en HTTP (test) et HTTPS (production avec <code>verify_ssl=true</code>)	OK

5.1.2 Tests de l’API

Pour évaluer le backend, on a exécuté une série de tests fonctionnels portant sur la validation des requêtes, les endpoints exposés et la persistance des données.

TABLE 5.2 – Tests de l’API WAF Backend

Test	Description	Résultat
Validation payload	Envoi d’un payload conforme : HTTP 200 et persistance en base	OK
Payload non conforme	Envoi d’un payload sans <code>agent_id</code> : HTTP 422 retourné avec détail des erreurs Pydantic	OK
Endpoint <code>/sources</code>	Consultation de la liste des serveurs WAF enregistrés	OK
Endpoint <code>/rules</code>	Consultation des règles actives par serveur avec filtrage	OK
Endpoint <code>/logs</code>	Consultation des logs d’audit avec filtres (sévérité, date, serveur, action) et pagination	OK
Authentification JWT	Accès à un endpoint protégé sans token : HTTP 401 retourné	OK
Persistance PostgreSQL	Vérification de la persistance des événements en base après réception du payload agent	OK

5.1.3 Tests de l’interface utilisateur

Afin de vérifier la qualité de l’interface utilisateur, on a testé les principales fonctionnalités liées à l’affichage, à la navigation et à l’authentification.

TABLE 5.3 – Tests du frontend Angular

Test	Description	Résultat
Affichage dashboard	Vérification du chargement des KPI et des anneaux statistiques	OK
Page Logs	Affichage correct des événements d'audit parsés depuis <code>modsec_audit.log</code> avec colonnes et badges sévérité	OK
Filtrage logs	Application des filtres (serveur, sévérité, date) et mise à jour de la liste	OK
Pagination	Navigaton entre pages de résultats (50 entrées / page)	OK
Authentification SSO	Connexion via Keycloak, récupération du token JWT et déconnexion	OK
Guards de routes	Accès à une page protégée sans connexion : redirection vers Keycloak	OK

5.1.4 Environnement de test

Afin de garantir une évaluation cohérente de la solution, les tests ont été réalisés dans un environnement proche du contexte réel d'exploitation. L'agent a été exécuté sur un serveur RHEL disposant d'une configuration ModSecurity active, l'API a été déployée sur OKD avec une base PostgreSQL dédiée, et le frontend Angular a été testé via un navigateur moderne dans un contexte d'authentification Keycloak. Cette organisation a permis de vérifier non seulement le bon fonctionnement unitaire des composants, mais aussi leur interopérabilité dans des conditions représentatives du projet.

L'environnement de test retenu poursuit trois objectifs principaux. Premièrement, il permet de valider la conformité fonctionnelle de chaque composant vis-à-vis des besoins définis au chapitre précédent. Deuxièmement, il facilite l'observation des points de défaillance potentiels lors des échanges entre les couches agent, backend et frontend. Enfin, il offre une base suffisante pour analyser la robustesse de la solution en présence de cas dégradés, tels qu'un payload incomplet, une erreur d'authentification ou un fichier de logs partiellement inaccessible.

5.1.5 Critères de validation retenus

L'évaluation de NovaGuard ne s'est pas limitée à une simple exécution technique des modules. On a défini plusieurs critères de validation permettant d'apprécier la qualité de la solution de manière plus globale.

- **Exactitude des données collectées** : l'agent doit remonter des informations fidèles sur les fichiers de configuration, les règles actives et les événements d'audit.
- **Fiabilité des échanges** : l'API doit accepter les données valides, rejeter les entrées incorrectes et assurer une persistance sans perte d'information.
- **Lisibilité des résultats** : le frontend doit restituer les informations de manière compréhensible et exploitable pour un analyste sécurité.
- **Robustesse opérationnelle** : le système doit continuer à fonctionner de façon partielle ou contrôlée même lorsqu'une erreur survient sur un composant donné.
- **Sécurité de bout en bout** : les mécanismes d'authentification, de contrôle d'accès et de transport sécurisé doivent rester cohérents sur l'ensemble de la chaîne.

Ces critères ont servi de fil conducteur lors de la lecture des résultats. Ils permettent de dépasser une logique binaire de type « succès / échec » et d'interpréter plus finement la valeur réelle de la plateforme dans un contexte d'usage professionnel.

5.1.6 Analyse qualitative des campagnes de test

Les campagnes menées montrent d'abord que l'architecture modulaire retenue constitue un atout pour la validation progressive du projet. En effet, chaque composant a pu être testé isolément avant de participer au flux global, ce qui a réduit le risque de confusion lors de l'identification des anomalies. Cette démarche a également facilité la localisation des défauts potentiels : lorsqu'un comportement inattendu apparaissait, il était possible de déterminer rapidement s'il provenait de la collecte, de la validation métier, de la persistance ou de la restitution visuelle.

Les tests mettent ensuite en évidence la cohérence globale des choix techniques effectués. L'agent assure une collecte structurée et suffisamment robuste, l'API fournit un cadre fiable pour la validation et le stockage, tandis que le frontend valorise correctement les données exposées. L'intérêt principal de cette campagne réside donc dans la démonstration du bon enchaînement entre des composants hétérogènes mais complémentaires, ce qui constitue un

enjeu central pour une plateforme de gouvernance WAF.

La stratégie de tests ayant été définie et appliquée aux différents composants, il convient maintenant d’analyser les résultats observés. Cette section présente les principaux enseignements issus des tests et met en évidence les bénéfices concrets de la solution.

5.2 Résultats obtenus

Les tests menés sur les différents composants ont permis de mesurer la robustesse globale de la solution et de valider son comportement dans un scénario d’utilisation réel.

5.2.1 Test de bout en bout

On a validé le flux complet de la chaîne NovaGuard selon le scénario suivant :

1. L’agent est lancé sur le serveur RHEL avec `collect_events=true`.
2. L’agent lit `modsec_audit.log`, parse les transactions et construit le payload JSON.
3. Le payload est envoyé via POST à `/api/waf/collect` sur OKD.
4. L’API valide le payload (Pydantic), persiste les événements en PostgreSQL et retourne HTTP 200.
5. Le frontend consomme `GET /api/waf/logs` et affiche les événements dans la page Logs avec les filtres et badges de sévérité.

Ce test bout en bout a été exécuté avec succès, validant l’ensemble de la chaîne agent → API → base de données → frontend.

5.2.2 Interprétation des résultats par composant

Les résultats obtenus sur l’agent montrent que la collecte locale constitue une base solide pour la supervision des serveurs WAF. Les tests de lecture de configuration, d’audit des règles et de parsing des logs ont confirmé que l’agent pouvait produire un état cohérent du serveur, même lorsque certaines contraintes système étaient introduites artificiellement. Cette observation est particulièrement importante, car la qualité de l’ensemble de la plateforme dépend directement de la fiabilité de cette première étape.

Du côté de l’API, les résultats confirment la pertinence du découplage entre validation, persistance et exposition des données. La gestion des erreurs de payload et le contrôle des accès montrent que le backend ne joue pas un

simple rôle de transit, mais qu'il agit comme un véritable point de contrôle centralisé. Cette capacité à filtrer et structurer les informations renforce la cohérence globale de NovaGuard, tout en réduisant le risque d'incohérences entre les données collectées et celles consultées par les utilisateurs.

Pour le frontend, les résultats illustrent un apport essentiellement opérationnel. L'interface permet de rendre visibles des informations qui, sans cela, resteraient dispersées dans plusieurs fichiers système ou réponses API. La réussite des scénarios liés à l'affichage des logs, au filtrage et à l'authentification démontre que la plateforme ne se limite pas à une simple centralisation technique : elle propose également un véritable support d'analyse pour les équipes sécurité.

5.2.3 Synthèse des bénéfices observés

Au-delà de la validation purement technique, les résultats mettent en évidence plusieurs bénéfices concrets pour l'exploitation de la solution.

TABLE 5.4 – Synthèse des bénéfices observés lors des tests

Axe	Bénéfice observé
Collecte	Centralisation rapide de l'état WAF depuis un serveur RHEL sans audit manuel complet
Validation	Détection immédiate des payloads invalides et rejet contrôlé des requêtes non conformes
Visualisation	Accès unifié aux informations critiques via une interface claire et exploitable
Traçabilité	Conservation structurée des événements d'audit dans PostgreSQL pour consultation ultérieure
Sécurité	Contrôle d'accès cohérent grâce à Keycloak, JWT et aux échanges sécurisés
Exploitation	Réduction du temps nécessaire pour comprendre l'état d'un serveur WAF ou analyser un événement

Cette synthèse montre que la valeur de NovaGuard ne repose pas uniquement sur l'automatisation. Elle réside également dans la capacité de la plateforme à transformer des informations techniques dispersées en éléments immédiatement utiles pour la décision, la supervision et l'investigation.

5.2.4 Limites de l'évaluation

Même si les résultats obtenus sont satisfaisants, l'évaluation réalisée dans le cadre de ce projet comporte certaines limites qu'il convient de souligner. D'abord, les tests ont été menés sur un nombre restreint de scénarios représentatifs, mais ils ne couvrent pas l'intégralité des cas extrêmes que l'on pourrait rencontrer dans un environnement de production à grande échelle. En particulier, les comportements sous forte volumétrie ou en présence d'un grand nombre simultané de sources WAF n'ont pas été étudiés de manière exhaustive.

Ensuite, l'évaluation du module d'intelligence artificielle s'est concentrée sur sa capacité à fournir une assistance contextualisée, et non sur une mesure quantitative de performance comparable à celle d'un modèle évalué sur un jeu de données académique. L'objectif était ici d'apprécier son intérêt pratique dans le cadre de l'analyse sécurité, plutôt que de produire un benchmark formel de type précision / rappel / F1-score.

Enfin, certains résultats restent dépendants des choix d'intégration retenus dans l'environnement cible, notamment la configuration Keycloak, les paramètres de déploiement OKD ou l'organisation des fichiers ModSecurity sur RHEL. Cela signifie que la reproductibilité est bonne sur une architecture proche, mais qu'une adaptation peut être nécessaire dans un autre contexte d'entreprise.

Conclusion

Les tests réalisés ont confirmé la fiabilité du flux de collecte bout en bout et la conformité de l'API aux spécifications définies. Les points d'amélioration identifiés constitueront la base des évolutions futures de NovaGuard.

CONCLUSION GÉNÉRALE

Ce projet de fin d'année a permis de concevoir et de développer **NovaGuard AI**, une plateforme de gouvernance WAF de nouvelle génération répondant à un besoin réel et stratégique de Sopra HR Software. En remplaçant la gestion manuelle et fragmentée de ModSecurity par une solution centralisée, intelligente et automatisée, NovaGuard apporte une valeur opérationnelle immédiate aux équipes sécurité et améliore la visibilité globale sur l'état des serveurs protégés.

Les résultats obtenus montrent que la solution repose sur une architecture cohérente et complémentaire. L'agent autonome déployé sur RHEL assure la collecte rapide et sécurisée des informations de configuration, des règles actives et des événements d'audit. Ces données sont ensuite transmises à une API centralisée robuste, déployée sur OKD, qui joue le rôle de point unique de validation, de persistance et d'exposition des informations. Le frontend moderne permet enfin une visualisation claire et actionnable de la posture de sécurité WAF, tandis que l'intégration d'un module d'intelligence artificielle, fondé sur un service externe complété par un mécanisme RAG local, enrichit l'analyse par des recommandations contextualisées et des propositions de règles adaptées.

Au-delà de son apport technique, ce projet a constitué une expérience particulièrement formatrice. Il a permis de consolider des compétences en développement backend avec Python et FastAPI, en architecture cloud autour d'OKD et de Kubernetes, en sécurité applicative avec WAF, ModSecurity, TLS et Keycloak, ainsi qu'en intelligence artificielle appliquée à l'analyse de sécurité. Cette dimension transversale renforce l'intérêt du projet, qui ne se limite pas à la mise en œuvre d'un outil, mais s'inscrit dans une véritable

logique de gouvernance et d'aide à la décision.

Plusieurs perspectives d'évolution peuvent prolonger ce travail. Il serait pertinent d'intégrer des tableaux de bord KPI pour suivre en temps réel les métriques de sécurité, d'enrichir le module RAG par une base de connaissances plus complète alimentée par les règles OWASP CRS et les incidents passés, d'automatiser davantage le déploiement de l'agent via des outils comme Ansible ou Helm, et de développer des mécanismes avancés de détection d'anomalies fondés sur l'apprentissage des patterns de trafic WAF. NovaGuard constitue ainsi une base solide et industrialisable pour la gouvernance WAF au sein de Sopra HR Software, avec un potentiel d'extension à l'échelle du groupe Sopra Steria.

BIBLIOGRAPHIE

- [1] OWASP Foundation. *ModSecurity Core Rule Set (CRS)* [en ligne]. OWASP, 2024. [consulté le 10 janvier 2025]. Disponible sur : <https://coreruleset.org/>
- [2] Ristic, Ivan. *ModSecurity Handbook*. Feisty Duck, 2010. ISBN 978-1-907117-02-2.
- [3] Ramirez, Sebastian. *FastAPI Documentation* [en ligne]. Tiangolo, 2024. [consulté le 15 janvier 2025]. Disponible sur : <https://fastapi.tiangolo.com/>
- [4] Google. *Angular 17 – Official Documentation* [en ligne]. Google, 2024. [consulté le 15 janvier 2025]. Disponible sur : <https://angular.io/>
- [5] OKD Community. *OKD – The Community Distribution of Kubernetes* [en ligne]. OKD, 2024. [consulté le 20 janvier 2025]. Disponible sur : <https://www.okd.io/>
- [6] Kubernetes Authors. *Kubernetes Documentation* [en ligne]. Cloud Native Computing Foundation, 2024. [consulté le 20 janvier 2025]. Disponible sur : <https://kubernetes.io/docs/>
- [7] Red Hat. *OpenShift Documentation* [en ligne]. Red Hat, 2024. [consulté le 20 janvier 2025]. Disponible sur : <https://docs.openshift.com/>
- [8] Rescorla, Eric. *The Transport Layer Security (TLS) Protocol Version 1.3*. IETF RFC 8446, 2018. Disponible sur : <https://datatracker.ietf.org/doc/html/rfc8446>
- [9] Red Hat. *Keycloak – Open Source Identity and Access Management* [en ligne]. Red Hat, 2024. [consulté le 20 janvier 2025]. Disponible sur : <https://www.keycloak.org/>

- [10] Lewis, Patrick ; Perez, Ethan ; Piktus, Aleksandra et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, vol. 33, p. 9459–9474.
- [11] OWASP Foundation. *OWASP Top Ten 2021* [en ligne]. OWASP, 2021. [consulté le 10 janvier 2025]. Disponible sur : <https://owasp.org/Top10/>
- [12] PostgreSQL Global Development Group. *PostgreSQL 15 Documentation* [en ligne]. PostgreSQL, 2024. [consulté le 25 janvier 2025]. Disponible sur : <https://www.postgresql.org/docs/>
- [13] Sopra HR Software. *Site officiel de Sopra HR Software* [en ligne]. Sopra Steria, 2024. [consulté le 5 février 2025]. Disponible sur : <https://www.soprahrsoft.com/>
- [14] Ollama. *Ollama Documentation* [en ligne]. Ollama, 2025. [consulté le 4 avril 2026]. Disponible sur : <https://ollama.com/>
- [15] Jiang, Albert Q. ; Sablayrolles, Alexandre ; Roux, Arthur et al. *Mistral 7B* [en ligne]. Mistral AI, 2023. [consulté le 4 avril 2026]. Disponible sur : <https://arxiv.org/abs/2310.06825>

ANNEXE A

ARBORESCENCE DU PROJET

Listing A.1 – Structure des dossiers du projet NovaGuard

```
novaguard/  
|-- agent/  
|   |-- novaguard_modsec_agent.py  
|   |-- config.json  
|   '-- requirements.txt  
|-- api/  
|   |-- main.py  
|   |-- models/  
|   |-- routers/  
|   '-- Dockerfile  
|-- frontend/  
|   |-- src/  
|       |-- app/  
|       '-- environments/  
|   '-- angular.json  
 '-- deployment/  
     |-- okd/  
     '-- docker-compose.yml
```

ANNEXE B

CONFIGURATION COMPLÈTE DE L'AGENT

Listing B.1 – Fichier config.json complet de l'agent NovaGuard

```
{
  "api_url":
    "https://waf.example.com/api/waf",
  "auth_token": "",
  "verify_ssl": true,
  "agent_id": "novaguard-agent-001",
  "collection_interval": 30,
  "retry_interval": 10,
  "max_retries": 3,
  "log_paths": [
    "/var/log/httpd/modsec_audit.log",
    "/var/log/httpd/modsec_audit.log-*"
  ],
  "collect_events": false,
  "max_events": 500,
  "mode": "auto",
  "output_dir": "./data",
  "state_file": "./state.json",
  "log_level": "INFO"
}
```

ANNEXE C

COMMANDES D'EXÉCUTION DE L'AGENT

Listing C.1 – Lancement standard de l'agent

```
python3 novaguard_modsec_agent.py -c config.json
```

Listing C.2 – Mode collecte unique pour diagnostic

```
python3 novaguard_modsec_agent.py -c config.json --run-once
```

ANNEXE D

CONFIGURATION RECOMMANDÉE EN PRODUCTION

Listing D.1 – Paramètres recommandés pour un déploiement production

```
{  
  "collect_events": true,  
  "verify_ssl": true,  
  "auth_token": "TOKEN_SECURE",  
  "collection_interval": 30,  
  "retry_interval": 10,  
  "max_retries": 3  
}
```

ANNEXE E

GUIDE DE DÉPANNAGE DE L'AGENT

TABLE E.1 – Tableau de dépannage de l'agent NovaGuard

Erreur observée	Solution recommandée
Erreur HTTP 401 / 403	Vérifier l'authentification et les droits côté API
Erreur TLS	Vérifier les certificats ou désactiver <code>verify_ssl</code> en environnement de test
Pas de règles	Vérifier <code>rules_paths</code> et les liens symboliques
Configs vides	Vérifier les permissions de lecture et les chemins de fichiers dans <code>config.json</code>
Pas d'événements SOC	Vérifier que <code>collect_events</code> est à <code>true</code>